



TU Clausthal

Bachelor Thesis

by

Steve Leonel, Yomi Mbiakop

**Beyond Accuracy: Measuring Intelligence in
Programming by Example**

1st Supervisor: **Prof. Dr. Christian Bartelt**

2nd Supervisor: **Dr. Sascha Marton**

26th August 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

Hiermit versichere ich, dass ich die Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe und dass alle Stellen dieser Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, als solche kenntlich gemacht wurden und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsstelle vorgelegt wurde.

Des Weiteren erkläre ich, dass ich mit der öffentlichen Bereitstellung meiner Abschlussarbeit in der Instituts- und/oder Universitätsbibliothek einverstanden bin.

Ich stimme hiermit außerdem einer softwaregestützten Plagiatsprüfung zu.

Clausthal-Zellerfeld, 26th August 2026

Location, Date

Steve Leonel, Yomi Mbiakop

Abstract

Programming by Example (PBE) lets users who cannot write code build programs simply by providing a handful of input-output examples, powering tools from spreadsheet automation to LLM-based coding assistants. Such users typically cannot read the generated program, leaving them little way to judge whether it captures their intent beyond re-checking it against the examples already given. Current evaluation practice mirrors this blind spot: PBE approaches are almost always compared by accuracy, the fraction of tasks whose output exactly matches the target. But accuracy is itself ambiguous, since it can mean checking a program against a reference or an output against the expected one, and either way a task collapses into a single binary success signal (BSS) that cannot separate a near miss from a complete failure or explain why a solved task worked.

This thesis develops four token-based program metrics, the Program Operation, Position, Sequence, and Edit Score (POS, PPS, PSS, PES), comparing a generated program directly to a reference program rather than only outputs. They are applied to two PBE approaches with fundamentally different representations: DeepCoder’s fixed domain-specific language and DreamCoder’s typed lambda calculus, which grows its own library of reusable components through a wake-sleep cycle. A shared evaluation procedure and purpose-built normalization pipeline make these two otherwise incomparable representations measurable on the same token vocabulary.

Across both approaches, the token-based metrics reveal a pattern the BSS alone cannot show: the POS is consistently higher than the other three metrics, even among solved tasks, meaning both approaches find the right operations more often than they combine them correctly. A task can be marked solved while its program bears little resemblance to the reference, and a task marked entirely wrong can already contain a structurally informative near-solution.

These findings show that a BSS can overstate or understate what a PBE approach has learned, letting researchers look past the pass/fail verdict. Solving a task is no longer the only question worth asking; how it got there, and how far it still has to go, matter just as much.

Zusammenfassung

Programming by Example (PBE) ermöglicht es Nutzern, die nicht programmieren können, Programme allein anhand weniger Input-Output-Beispiele zu erstellen, von der Automatisierung in Tabellenkalkulationen bis zu modernen LLM-basierten Programmier-Assistenten. Solche Nutzer können das erzeugte Programm meist nicht selbst lesen, weshalb ihnen kaum eine Möglichkeit bleibt zu beurteilen, ob es ihre Absicht trifft, außer es erneut an denselben Beispielen zu prüfen, die sie bereits gegeben haben. Die gängige Evaluationspraxis spiegelt genau diesen blinden Fleck wider: PBE-Ansätze werden fast ausschließlich über accuracy verglichen, den Anteil der Aufgaben, deren Ausgabe exakt mit dem Ziel übereinstimmt. Accuracy selbst ist aber mehrdeutig, da sie entweder ein erzeugtes Programm gegen ein Referenzprogramm oder eine erzeugte Ausgabe gegen die erwartete prüfen kann, und so oder so schrumpft jede Aufgabe auf ein einziges binäres Erfolgssignal (BSS) zusammen, das einen knappen Fehlschlag nicht von vollständigem Versagen unterscheidet und nichts darüber aussagt, warum eine gelöste Aufgabe funktioniert hat.

Diese Arbeit entwickelt vier tokenbasierte Programmmetriken, den Program Operation, Position, Sequence und Edit Score (POS, PPS, PSS, PES). Statt nur Ausgaben zu vergleichen, stellen sie ein erzeugtes Programm direkt einem Referenzprogramm gegenüber, angewendet auf zwei PBE-Ansätze mit grundverschiedener Programmrepräsentation: DeepCoders feste domänenspezifische Sprache (DSL) und DreamCoders typisierten Lambda-Kalkül, der seine eigene Bibliothek wiederverwendbarer Bausteine über einen Wake-Sleep-Zyklus erweitert. Ein einheitliches Evaluationsverfahren und eine eigens entwickelte Normalisierungspipeline machen diese beiden sonst unvergleichbaren Programmrepräsentationen auf demselben Token-Vokabular messbar.

Bei beiden Ansätzen zeigen die tokenbasierten Metriken ein Muster, das dieses BSS allein nicht sichtbar machen kann: Der POS liegt selbst unter den gelösten Aufgaben durchgängig höher als die drei übrigen Metriken, das heißt, beide Ansätze finden die richtigen Operationen häufiger, als dass sie diese korrekt kombinieren. Eine Aufgabe kann als gelöst gelten, obwohl ihr Programm kaum dem Referenzprogramm ähnelt,

und eine als vollständig falsch bewertete Aufgabe kann bereits eine strukturell aufschlussreiche Beinahe-Lösung enthalten.

Diese Befunde zeigen: Ein BSS kann überschätzen, was ein PBE-Ansatz gelernt hat, oder es unterschätzen, und lässt Forschende und Praktiker so über das reine Bestehen-oder-nicht-Urteil hinausblicken. Ob eine Aufgabe gelöst wurde, ist damit nicht mehr die einzige Frage, die zählt; wie ein Ansatz dorthin gelangt ist und wie weit er noch entfernt ist, zählt genauso.

Acknowledgements

I would like to thank my two examiners, Prof. Dr. Christian Bartelt and Dr. Sascha Marton, for evaluating this thesis.

I am especially grateful to my supervisor, Janis Zenkner, for his guidance and feedback throughout the course of this work.

I would also like to thank all the other professors at TU Clausthal who accompanied me throughout my studies and shared their valuable knowledge with me.

A heartfelt thank you goes to my mother, Kemta Martine, for her unconditional support.

I would also like to thank Dr. Leon Kellerhals, who supported me greatly throughout my studies.

Last but not least, I thank my entire family and all my friends for their support and patience throughout my studies.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Aim of the Thesis	3
1.4	Research Question	4
1.5	Contribution of the Thesis	4
2	Related Work	6
2.1	Solving Programming by Example Tasks	6
2.2	Evaluating PBE and Code-Generation Approaches	9
2.3	The Evaluated Approaches: DeepCoder and DreamCoder	11
2.3.1	DeepCoder	11
2.3.2	DreamCoder	12
3	Theoretical Background	15
3.1	Program Synthesis	15
3.2	Programming by Example	17
3.3	Accuracy as an Evaluation Metric	18
4	Methodology	21
4.1	Data Conversion	21
4.2	Token-based Program Metrics	22
4.2.1	Program Operation Score	23
4.2.2	Program Position Score	24
4.2.3	Program Sequence Score	24
4.2.4	Program Edit Score	25
4.3	Comparison and Interpretation of the Metrics	26

5	Evaluation	28
5.1	Task Domain	28
5.2	Compared Configurations	30
5.3	Hyperparameters and Reproducibility	31
5.4	Reporting Convention	31
6	Results and Discussion	33
6.1	Performance	33
6.2	Quality of the Generated Programs	34
6.2.1	Program Operation Score	35
6.2.2	Program Position Score	36
6.2.3	Program Sequence Score	37
6.2.4	Program Edit Score	38
6.3	Connection between Performance and Program Quality	40
6.3.1	The POS Advantage: A Recurring Pattern	40
6.3.2	All Tasks versus Solved Tasks	42
6.3.3	Best-Effort Candidates on Unsolved Tasks	43
6.3.4	DeepCoder versus DreamCoder	45
6.3.5	Worked Examples	46
6.4	Limitations	50
7	Conclusion	52
7.1	Future Work	53
	References	55

List of Figures

6.1	DeepCoder: solve rate, exact-match fraction, and the four token-based metrics, without vs. with training ($gas = 1,000$, mean over all 100 tasks).	41
6.2	DreamCoder: solve rate and the four token-based metrics by configuration (500 training tasks, 3 iterations, mean over all 99 tasks).	41
6.3	DeepCoder: the four token-based metrics without vs. with training, averaged over solved tasks only.	43
6.4	DeepCoder: the four token-based metrics for the best-effort candidates of unsolved tasks, without training vs. with training.	45
6.5	DreamCoder: the four token-based metrics by configuration, averaged over solved tasks only.	46

List of Tables

2.1	Representative PBE-solving approaches, contrasted by how they represent programs and what guides their search.	7
3.1	The same transformation expressed through three different specification types.	15
3.2	Two candidate programs, “return the maximum” and “return the last element,” both satisfy Examples 1 and 2. Only Example 3 disambiguates between them.	17
3.3	The running example task used throughout this thesis: doubling and sorting a list of integers.	19
4.1	The example used throughout this chapter: doubling and sorting a list of integers.	22
4.2	The four token-based metrics, what each one captures, and its value on the running example from Section 4.2.	26
5.1	A list processing task with five input-output examples, together with a reference program of length two that satisfies all of them.	29
6.1	Solve rate (BSS) across all five configurations.	33
6.2	POS, averaged over all tasks and over only the solved tasks.	35
6.3	PPS, averaged over all tasks and over only the solved tasks.	36
6.4	PSS, averaged over all tasks and over only the solved tasks.	37
6.5	PES, averaged over all tasks and over only the solved tasks.	38
6.6	Best-effort structural metrics on unsolved tasks only, averaged over all unsolved tasks in that seed (tasks without a stored candidate score 0; the count in parentheses is the number of those unsolved tasks with a stored candidate).	44

6.7	All five given examples of the task from the first example, with the output expected by the reference program and the actual output of the best candidate found.	47
6.8	All five given examples of the task from the second example, with the output expected by the reference program and the actual output of the found solution.	48
6.9	All five given examples of the task from the third example. Since the found program ignores its input entirely, only the lengths of the two input lists and the integer are given here, not the actual values.	49

1 Introduction

1.1 Motivation

Programming by Example (PBE) is an approach to program synthesis (PS) in which a desired program is derived from a set of input-output examples [10, 2]. Instead of writing a program manually or formulating a formal specification, the user describes the desired behavior through examples or demonstrations [14, 9]. A PBE approach then attempts to find a program that satisfies these examples and generalizes to new inputs [10, 16]. This makes PBE especially relevant for automating tasks such as data cleaning, string transformations, and the processing of tabular data, since it allows users without advanced programming skills to specify what they want through a few examples rather than writing code [14, 9, 2].

A widely used example of PBE in practice is Microsoft Excel’s Flash Fill feature, which infers a string-transformation program from a handful of example cells the user has filled in by hand, for instance splitting a full name into separate first- and last-name columns, and then applies the inferred program to the remaining rows [9]. Flash Fill illustrates both the appeal and the risk of PBE at once: the user only ever sees the transformed values, not the program that produced them, so a plausible-looking but subtly wrong transformation can go unnoticed until it silently corrupts a row the user never checked by hand.

PS restricts the space of candidate programs it searches through a domain-specific language (DSL), a language that fixes which operations, constants, and program structures are permitted for a given class of tasks [9, 2]. PBE is the special case of PS in which the specification takes the form of input-output examples rather than, say, a formal condition or natural-language description.

PBE approaches can be realized in different technical ways: classical approaches are frequently based on symbolic search in a DSL, while more recent approaches also include neural and Large Language Model (LLM)-based models [10, 5, 16]. Regardless of the concrete synthesis approach, however, the central question remains the same: whether

the generated program actually captures the intended transformation, or merely happens to satisfy the examples it was given. This question matters in practice, because PBE approaches are increasingly aimed at users who cannot inspect the generated program themselves and must trust the approach’s output at face value [14, 9].

This makes evaluation a fundamental problem in PBE. A generated program can satisfy the given input-output examples without actually representing the intended rule. Because examples usually constitute an incomplete specification, several different programs can be consistent with the observed examples yet behave differently on new inputs [10, 26].

This uncertainty is directly reflected in how the performance of PBE approaches is evaluated in practice.

1.2 Problem Statement

Many existing evaluation approaches assess PBE approaches using accuracy metrics. Typically, this checks whether the output produced by the approach exactly matches the expected output [16]. If this is the case, the task is considered correctly solved; otherwise it is judged incorrect [3, 16].

This binary evaluation is simple and intuitive, but it falls short in many cases. A generated solution can be partially correct even if it does not exactly match the expected output. Suppose, for example, that an approach is meant to transform the input [3, 1, 2] by doubling each element and sorting the result, so that the expected output is [2, 4, 6]. If the approach instead produces [3, 4, 5], having correctly sorted the input but added 2 to each element instead of doubling it, this binary judgment marks the solution as completely wrong. At the same time, the output shows that the approach has captured part of the intended transformation: it identified that the input needed to be sorted and applied some operation to each element, just not the correct one.

As the example illustrates, this binary success signal (BSS) reduces every solution to a single judgment and cannot express how close a wrong solution actually is to the target. An arbitrary, unrelated program and a program that gets almost everything right receive the exact same score. This makes it difficult to tell whether an approach fails because it misunderstands a task entirely or because it comes structurally close to the correct solution but makes a small, systematic mistake.

This limitation is not confined to a single sorting example. Two generated programs can produce the same correct output on the observed examples while arriving at it in

entirely different ways, one of them explicitly reproducing the intended transformation, the other combining several unrelated operations that happen to cancel out on exactly the examples it was tested against. An evaluation that only inspects the output cannot distinguish these two cases, because it never looks at the program that produced it. The same asymmetry applies in the other direction: a program can be judged entirely wrong on the output level while it already contains most of the correct transformation, missing it only by a small, identifiable step. In both directions, whether an approach genuinely captured the intended rule or merely got lucky, and how close a failed attempt actually came, are questions that an output-level judgment by construction cannot answer.

The comparisons above are therefore possible at two different levels. One can compare the produced output to the expected output, as above, or one can compare the generated program itself to a reference program that is known to produce the correct behavior. Comparing at the program level makes it possible to see *how* an approach arrived at its result, not only whether the final output happens to be right, and can reveal partially correct or structurally close solutions that an output-level judgment alone would report as uniformly wrong. This is not free of trade-offs, since a program-level comparison requires a reference program to compare against and depends on how differently two approaches represent programs in the first place; Chapter 6 returns to these limitations.

1.3 Aim of the Thesis

The aim of this thesis is to develop and evaluate metrics that go beyond a BSS by measuring how structurally close a generated program is to a correct solution, at the level of the program itself rather than only its output. Instead of asking only whether a task was solved, such metrics ask how much of the correct solution a generated program actually captures, and in what respect it deviates from it.

The thesis focuses on partial correctness as the central evaluation aspect that follows from this gap: whether a generated program, while not fully correct, nonetheless contains meaningful parts of the expected solution. This is intended to enable a more fine-grained evaluation of PBE approaches, one that can show in which cases a BSS provides only an incomplete picture of an approach's actual performance.

Condensed into a single, precisely formulated question, this aim becomes the starting point for the entire thesis.

1.4 Research Question

The central research question of this thesis is as follows:

What additional insights into the performance of PBE approaches can be gained through evaluation metrics beyond binary success signals?

To answer this research question, the following aspects are considered in particular:

- Which aspects of PBE approach behavior remain hidden by a BSS?
- What additional information can be gained through the analysis of partial correctness?
- In which cases do program-level metrics provide a more informative explanation of why a task was solved or not solved?

To answer these questions, this thesis makes several concrete contributions.

1.5 Contribution of the Thesis

The contributions of this thesis are as follows:

- We transfer established concepts from edit-distance and sequence-similarity metrics for the first time uniformly to tokenized *programs* rather than their outputs, so that partial correctness becomes comparable independently of the respective DSL or search method.
- We demonstrate the transferability of this approach on two PBE approaches with fundamentally different program representations. Since a direct token comparison between DeepCoder’s DSL and DreamCoder’s lambda calculus would otherwise not be possible, a purpose-built mapping onto a shared vocabulary was required for this.
- We quantify how strongly a BSS overstates actual model performance, and in doing so identify a recurring error pattern: both approaches find the correct operations more often than they combine them correctly.

- We show that even failed search attempts can contain usable information, by extending DeepCoder’s search so that a score beyond the value 0 becomes possible for a subset of unsolved tasks as well.

Realizing these contributions requires building up the necessary background first. Chapter 2 situates the thesis within existing approaches to solving and evaluating PBE tasks to clarify what specifically remains unaddressed. Chapter 3 then establishes the theoretical foundations of PS, PBE, and evaluation metrics on which the rest of the argument builds. Chapter 4 introduces the four token-based metrics developed to close the identified gap and describes how they were made comparable across DeepCoder and DreamCoder despite their differing program representations. Chapter 5 fixes the concrete conditions under which both approaches are compared. Chapter 6 presents and discusses the resulting experiments together, including the limitations of this work. Chapter 7 summarizes the findings, answers the research question, and outlines directions for future work.

2 Related Work

Where this thesis fits into the existing literature follows from what it deliberately does not do: propose a new solving methodology. Instead, it investigates whether token-based metrics give more insight into an approach’s performance than its BSS alone, so the literature below is organized along exactly this split: Section 2.1 groups existing approaches by *how* they solve PBE tasks, and Section 2.2 groups existing approaches by *how* they evaluate the resulting solutions. Section 2.3 then introduces DeepCoder and DreamCoder in detail, the two PBE approaches to which the metrics developed in this thesis are applied.

2.1 Solving Programming by Example Tasks

Approaches to solving PBE tasks differ mainly in how they guide the search for a satisfying program. Some rely on predefined rules and decomposition, others on neural predictions or on feedback from executing partial programs, and a more recent line simply prompts a pretrained language model directly. Table 2.1 summarizes this by contrasting how a representative approach from each group represents programs and what guides its search, before the remainder of this section characterizes each in more detail.

Classical Search-Based Approaches. Gulwani et al. [10] give a general overview of PS and PBE. A well-known practical instance is FlashFill, which derives string transformations for spreadsheet columns from a handful of examples [9]. FlashMeta generalizes this idea into a reusable synthesis framework: it decomposes the specification of an output into specifications for its sub-expressions, so that domain-specific synthesizers for many string, format, and layout transformations can be constructed from shared building blocks rather than written from scratch [19]. Building on this line of work, FlashFill++ extends the underlying DSL to cover substantially more realistic spreadsheet transformations while retaining synthesis speeds comparable to the original FlashFill [2]. Pro-

Table 2.1: Representative PBE-solving approaches, contrasted by how they represent programs and what guides their search.

Approach	Program representation	Guidance mechanism
FlashMeta [19]	DSL (string/format transformations)	Symbolic decomposition, no learned component
RobustFill [5]	DSL (string transformations)	Seq2seq network generates the full program directly
DeepCoder [1]	Fixed DSL (list processing)	DFS guided by a neural attribute predictor
Write-Execute-Assess [6]	DSL (graphics, string editing)	Policy and value networks guided by execution state
DreamCoder [7]	Evolving typed lambda-calculus library	Wake-sleep cycle: library learning and neural recognition
Codex [3]	General-purpose Python, no DSL	Pretrained LLM, repeated sampling (pass@k)

gramming by Demonstration is a closely related paradigm: Lau et al. [14] show that programs can also be derived from a user’s demonstrated actions rather than explicit input-output pairs, representing the entire set of programs consistent with the demonstrations so far as a version space that is updated incrementally as more demonstrations arrive.

Neurally Guided Search. RobustFill trains a sequence-to-sequence model to predict a complete program directly from a set of input-output examples [5]. Devlin et al. compare four model variants that progressively add attention between the input-output encoder and the program decoder, finding that the attention-based variants substantially outperform a non-attentional baseline and, notably, remain robust when the given examples contain noise such as typos. DeepCoder instead keeps the search itself unchanged and uses a neural network only to predict, from the given examples, which operations of a fixed DSL a solving program is likely to use [1].

Execution-Guided and Decomposition-Based Synthesis. A different line of work focuses less on the search space itself and more on how the synthesis process unfolds step by step. Ellis et al. [6] equip a neural search process with a read-eval-print loop that immediately executes partially written programs, so that a policy model proposing the next piece of code and a value model assessing the partial program’s prospects can both condition on the resulting execution state rather than on syntax alone. This

addresses a basic difficulty of program search: small syntactic changes can produce large, unpredictable semantic changes, which a purely syntactic search cannot anticipate. Building on this idea of decomposing synthesis into smaller steps, Shi et al. [24] propose ExeDec, which interleaves two neural models within a beam search over execution states rather than over raw code: a subgoal model that predicts the intermediate program state the next part of the program should reach, and a synthesizer model that generates code toward that predicted state. Zenkner et al. [30] revisit this framework and compare it against REGISM, their own ablated variant that relies solely on repeated execution-guided synthesis without explicit decomposition. Explicit decomposition helps with generalizing to longer programs and novel concept combinations, but the ablated, purely execution-guided variant frequently matches or even exceeds the decomposition-based approach on other measures. Decomposition, in other words, may not be the primary driver; iterative execution guidance likely accounts for much of ExeDec’s performance instead. DreamCoder, the second approach evaluated in this thesis, similarly grows its own library of reusable program components across repeated learning cycles [7].

Program Synthesis using LLMs. Chen et al. [3] investigate, with Codex, how well large language models generate Python functions from natural-language docstrings, evaluated via the *pass@k* metric detailed in Section 2.2. A single sample from their 12-billion-parameter model solves 28.8% of their HumanEval benchmark, while sampling 100 candidates per problem and keeping any one that passes raises this to 70.2%. A model, it turns out, can often find a correct solution without reliably producing it on the first attempt. Li and Ellis [16] examine specifically whether LLMs solve classic PBE tasks, that is, tasks specified purely through input-output examples without any accompanying natural-language description. Their results show that LLMs can reach high performance after fine-tuning, particularly when test tasks resemble the training distribution, but that they still struggle with tasks outside this distribution (out-of-distribution generalization) and with combining known concepts in new ways (systematic generalization). More recently, Wei et al. [27] introduce CodeARC, an interactive benchmark in which agents iteratively refine candidate programs based on execution feedback rather than being judged on a single static attempt; even under this more informative evaluation, current LLMs solve fewer than 53% of tasks. The difficulty Li and Ellis [16] observed, then, is not simply an artifact of one-shot evaluation.

The guiding mechanism varies across all four groups. What stays constant is the criterion for judging the result: almost every approach discussed above is compared to

others by whether its output matches the expected outcome.

DeepCoder and DreamCoder, the two approaches evaluated in this thesis, come from two different groups among these, neurally guided search over a fixed DSL and execution-guided synthesis over a growing library, respectively. The metrics developed in this thesis can therefore be tested against two structurally different solving approaches at once, not merely a single search mechanism. How well an approach can even be assessed on this basis, however, depends on how it is evaluated.

2.2 Evaluating PBE and Code-Generation Approaches

Existing evaluation approaches for PBE and code generation differ in how much they extract beyond a per-task pass/fail verdict. Some stick to purely functional correctness, others examine failure behavior in more targeted ways, and a third group compares the generated artifact itself rather than only its execution result.

Functional-Correctness Based Evaluation. Chen et al. [3] popularize *pass@k* for code generation: a problem counts as solved if at least one of k independently sampled programs passes the given test cases. Because it only requires one success among several attempts, *pass@k* separates a model’s raw generative capability from the quality of a later selection step, but it says nothing about the many attempts that fail. APPS evaluates generated code against held-out test cases in the same functional-correctness spirit, additionally reporting an *Accuracy with Partial Credit* variant that counts the proportion of test cases passed per task rather than requiring all of them [11]. Hendrycks et al. show the practical relevance of this distinction empirically: for GPT-2 1.5B, plain accuracy averages 2.08%, while Partial-Credit accuracy averages 9.00%. A purely binary evaluation, in other words, badly understates how often the model actually produces partially correct code. Compute-matched evaluation is a related concern: Sesterhenn et al. [23] show that reported performance differences between approaches can partly be explained by unequal compute budgets rather than by the approaches themselves. Reporting what was measured is not enough for a fair evaluation; the resource conditions under which it was measured matter too.

Beyond a Single Aggregate Score. Several works argue that even graded functional-correctness scores obscure important failure patterns. CodeBLEU combines n-gram similarity with syntactic and semantic code features to address specific weaknesses of

BLEU and exact match when applied to code [21]. CheckList takes a complementary, behavioral-testing perspective, showing across several NLP tasks that specific, systematic failure modes remain invisible when only a single aggregated accuracy value is reported [22]. Most recently, RACE evaluates generated code along four separate dimensions, readability, maintainability, correctness, and efficiency, and finds that even advanced LLMs satisfy customized, real-world requirements far less reliably than plain correctness scores would suggest [31]. These works motivate reporting more than one number per approach.

Output-Based Similarity Metrics. A separate line of work measures how close an incorrect output is to the target rather than only whether it is exactly right. Edit-distance metrics such as the Levenshtein distance count the minimum number of insertions, deletions, and substitutions needed to transform one sequence into another [15]; Damerau [4] extends this notion with adjacent-character transpositions to better capture common typing errors. Longest Common Subsequence-based methods measure structural similarity differently: Lin and Och [17] use LCS overlap to score machine-translation output, an idea this thesis adapts in Chapter 4 by scoring the longest *contiguous* common token block instead, to make visible which uninterrupted parts of a program’s structure are preserved. Further established metrics operate at yet other levels: BLEU scores n-gram precision against one or more references [18], and Kendall’s coefficient scores rank correlation between two orderings [13]. Closer to program synthesis specifically, Song et al. [25] show that Abstract Syntax Tree edit distance captures code structure more precisely than purely token-level metrics such as BLEU, since it operates on the parsed program structure rather than its surface tokens. This thesis deliberately builds on token-based rather than tree-based comparison, since the same token vocabulary can be applied uniformly across DeepCoder’s and DreamCoder’s structurally different program representations, described next, without requiring a shared tree grammar between them.

None of these approaches, neither the purely functional ones nor the multi-dimensional ones nor the output-based similarity metrics, compares the generated program itself to a reference, exactly the gap the token-based metrics developed in this thesis address.

2.3 The Evaluated Approaches: DeepCoder and DreamCoder

To examine whether token-based metrics generalize across approaches, this thesis applies them to two existing PBE approaches that represent and search for programs in entirely different ways. DeepCoder searches a fixed, hand-written DSL using a neural network that predicts which operations a solving program likely contains [1]. DreamCoder instead represents programs as typed lambda-calculus expressions and grows its own library of reusable components over repeated learning cycles [7]. This section describes each in the detail required to later explain how their token vocabularies were made comparable.

2.3.1 DeepCoder

DeepCoder instantiates a general framework the authors call Learning Inductive Program Synthesis (LIPS), which factors program synthesis into four components: a DSL together with an attribute function, a procedure for generating training data, a machine learning model, and a search procedure that the model’s predictions guide [1].

DSL and Attributes. DeepCoder’s DSL is loosely inspired by query languages such as SQL, in which high-level functions are applied in sequence to manipulate data. A program is a sequence of function calls, where each call initializes a fresh variable from the inputs or from previously computed variables, and the program’s output is the value of the last variable. The DSL contains 34 functions in total: first-order list functions such as HEAD, LAST, TAKE, DROP, ACCESS, MINIMUM, MAXIMUM, REVERSE, SORT, and SUM, together with higher-order functions, MAP, FILTER, COUNT, ZIPWITH, and SCANL1, that each take one of several fixed lambda functions (for example $(*2)$, (>0) , or MIN) as an argument. LIPS defines an *attribute function* that maps a program to a fixed-length binary vector recording whether each of the 34 functions occurs in it anywhere; this attribute vector is the interface between the neural network and the search procedure, since the network only needs to predict which operations a program likely uses, not their exact arrangement.

Data Generation and Model. Training data is generated by enumerating DSL programs and heuristically pruning those with detectable redundancies, such as a variable that does not affect the output or a shorter behaviorally equivalent program. For each retained program, input values are sampled within a bounded integer range chosen so

that execution stays well-defined, and the corresponding outputs are computed by running the program. The resulting input-output examples and binary attribute vectors form the supervised training signal. The network itself has an encoder-decoder structure: the encoder represents each input-output example by one-hot encoding its type (single integer or array), embedding every integer into a learned 20-dimensional vector, and passing the concatenated representation through three hidden layers of 256 sigmoid units each; representations of several examples for the same program are then averaged. The decoder multiplies this pooled representation by a learned matrix to produce one logit per DSL function, trained with a cross-entropy loss so the outputs can be read as independent marginal probabilities that each function occurs in the target program.

Search. These predicted probabilities are used to guide a depth-first search (DFS) that tries DSL functions at each search step in order of predicted likelihood, and a *sort and add* enumeration scheme that starts from the most probable functions and progressively admits less probable ones only once the current active set fails to yield a program within the search budget. The same predictions can also guide existing solvers from the programming-languages community, including the SMT-based tool Sketch and the enumerative-with-deduction tool λ^2 , by restricting the operations these solvers consider to the current active set. Balog et al. [1] report that this neural guidance yields an order-of-magnitude speedup over unguided search baselines and over a comparable RNN-based approach, allowing DeepCoder to solve problems of a difficulty comparable to the simplest problems on programming-competition websites. The DSL and its production rules, however, never change during this process. DeepCoder’s neural network only reorders an otherwise fixed search space.

DeepCoder’s vocabulary therefore remains static throughout the entire search and training process: a program space that is fixed from the outset, differing only in the order in which it is searched.

2.3.2 DreamCoder

DreamCoder frames program induction as Bayesian inference and represents programs as expressions of a polymorphically typed lambda calculus rather than sequences over a fixed DSL [7]. Its central object is a *library* L , a growing set of primitives and learned abstractions that together define a prior $P[\rho \mid L]$ over programs ρ . Learning proceeds in repeated *wake-sleep* cycles, each consisting of one wake phase and two interleaved sleep

phases, that jointly approximate the following three updates for a set of tasks X :

$$\rho_x = \arg \max_{\rho} P[\rho \mid x, L] \propto P[x \mid \rho] P[\rho \mid L] \quad (\text{Wake})$$

$$L = \arg \max_L P[L] \prod_{x \in X} \max_{\rho \text{ a refactoring of } \rho_x} P[x \mid \rho] P[\rho \mid L] \quad (\text{Sleep: Abstraction})$$

$$\text{train } Q(\rho \mid x) \approx P[\rho \mid x, L] \quad (\text{Sleep: Dreaming})$$

where $P[L]$ is a description-length prior that favors compact libraries and $P[x \mid \rho]$ is 1 if program ρ reproduces the input-output examples of task x and 0 otherwise. The wake phase does not search over all programs exhaustively; it is restricted to candidates the recognition model $Q(\rho \mid x)$ already considers likely, which is what makes the search tractable.

Wake Phase. During waking, DreamCoder searches for a program that solves each task in the current batch, guided by a neural *recognition model* $Q(\rho \mid x)$ that proposes candidate programs in decreasing order of predicted probability until either a solution is found or a timeout is reached; to tolerate cases where several candidate programs solve a task, the $k = 5$ highest-scoring solutions are kept for use in the following sleep phases rather than only the single best one.

Sleep: Abstraction (Consolidation). This thesis follows Ellis et al.’s own terminology and refers to the abstraction phase as *Consolidation*. Its objective is to grow the library L by refactoring, that is, rewriting, the programs found during waking into semantically equivalent but syntactically different forms, and incorporating whichever resulting subexpressions best compress the solutions found so far into the library as new primitives. Because a program admits an astronomically large number of possible refactorings, up to roughly 10^{14} for a small example given in the original paper, DreamCoder represents the set of refactorings compactly using a version-space-algebra-like data structure rather than enumerating them individually, bounding the number of evaluation steps a refactoring may take to keep this representation tractable. In this way, Consolidation can discover a reusable primitive such as `map` from only a handful of solved list-processing tasks and add it to the library for later reuse.

Sleep: Dreaming (Recognition). The second sleep phase, referred to here as *Recognition*, trains the neural recognition model $Q(\rho \mid x)$ used during waking. Training pairs

are drawn in equal parts from *replays*, tasks the system already solved during waking, and *fantasies*, synthetic tasks obtained by sampling a random program from the current library L and executing it. Fantasies are essential for data efficiency, since only 100 to 200 real tasks are typically available per domain, far too few to train a high-capacity network on their own, whereas the library can generate an effectively unlimited stream of synthetic training data once it has learned a handful of domain-relevant primitives. Training uses a maximum-a-posteriori objective, which additionally breaks ties among semantically equivalent but syntactically distinct programs by concentrating the model’s probability mass on a single canonical form.

Effect and Comparison to DeepCoder. Consolidation and Recognition address complementary sources of search cost: higher-level library primitives let a solution be expressed with fewer function calls, reducing how *deep* the search tree needs to go, while the recognition model down-weights unlikely expansions, reducing how *wide* the search needs to be at each step; Ellis et al. [7] note that overall search difficulty scales roughly as breadth to the power of depth, so that either mechanism alone can already yield large gains. The practical effect can be substantial: in the original paper’s own list-processing example, a program that sorts a list requires 32 function calls when expressed only in DreamCoder’s initial primitives, which the authors estimate would take in excess of 10^{72} years to find by brute-force enumeration, but only 5 function calls once six rounds of Consolidation have enriched the library, solvable in under 10 minutes of search [7]. Here lies the central structural contrast to DeepCoder. DeepCoder’s DSL and its 34 functions stay fixed throughout search; only the network’s guidance changes. DreamCoder’s library, and therefore its very vocabulary of programs, evolves across the run instead.

Because DeepCoder’s fixed DSL and DreamCoder’s evolving lambda-calculus library share no common vocabulary, their programs cannot be compared token-by-token without further work. Chapter 4 describes the mapping that makes this comparison possible.

This missing shared vocabulary constitutes the central technical obstacle: it must be resolved before the metrics developed in this thesis can be applied to both approaches at all. Before that, however, comes the theoretical framework in which these metrics are embedded: PS, PBE, and the accuracy metric, at whose limits this thesis begins.

3 Theoretical Background

Three concepts underpin everything that follows: PS, its special case PBE, and the accuracy metric used to evaluate both. Section 3.1 introduces PS and the DSLs it commonly searches over. Section 3.2 narrows this down to PBE, the specific form of PS this thesis is concerned with. Section 3.3 then introduces accuracy, the standard metric used to evaluate PS and PBE approaches, and defines a running example task that is reused throughout the following chapters to illustrate the metrics this thesis develops.

3.1 Program Synthesis

Gulwani et al. [10] define PS as the automatic generation of a program from a specification of the desired behavior. Instead of writing the program by hand, a user or developer describes what the program should do, and a synthesis approach searches for a program that satisfies this description. Such a specification can take several forms, for example a formal logical condition, a natural language description, or a set of input-output examples [10, 16]. Table 3.1 illustrates these forms on the same target transformation, squaring an integer.

Table 3.1: The same transformation expressed through three different specification types.

Specification type	Example
Formal condition	$\forall x. P(x) = x \cdot x$
Natural language	“Square the input number.”
Input-output examples	$2 \rightarrow 4, 3 \rightarrow 9, 4 \rightarrow 16$

Each specification type comes with a trade-off. A formal condition is unambiguous but can be as much effort to write as the program itself, while a natural language description is easy to write but often underspecified or open to multiple readings [10]. Input-output examples sit between these two extremes. They are simple to provide, and

they still ground the specification in concrete, checkable behavior. This trade-off is a central reason why PBE has become a widely used PS paradigm in practice.

Domain-Specific Languages. The choice of specification also shapes how a synthesis approach searches for a program. Rather than searching over an unrestricted, Turing-complete programming language, PS approaches typically restrict the space of candidate programs to a DSL: a language that fixes which operations, constants, and program structures are permitted for a given class of tasks [9, 2]. This DSL need not stay fixed throughout the entire search, DreamCoder, for instance, continually extends its own DSL with newly learned building blocks [7], nor does it need to exist at all: LLM-based approaches such as Codex forgo a DSL entirely and generate code directly in a general-purpose programming language such as Python [3]. A DSL for the squaring task from Table 3.1 might contain little more than a single multiplication operation; the list-processing domain used later in this thesis needs a richer vocabulary, with operations such as `SORT` to sort a list or `MAP` to apply a function to every element of a list.

This restriction is what makes searching for a correct program tractable in the first place. Consider a DSL with k available operations and a target program of length n , where each of the n positions in the program is filled by one of the k operations. Without any further restrictions, the number of syntactically possible programs of this length grows as k^n . For a DSL with $k = 20$ operations and a program of length $n = 5$, this already amounts to $20^5 = 3,200,000$ candidate programs. Restricting the DSL to a general-purpose language with, say, $k = 2,000$ available operations and identifiers increases this number to $2,000^5 = 3.2 \times 10^{16}$, ten billion times larger for the same program length. This is why PS approaches use small, task-tailored DSLs instead of general-purpose languages such as Python or Java: an exhaustive search over the latter quickly becomes computationally infeasible [10]. At the same time, a DSL that is too restrictive limits which transformations can be expressed at all, so, as Cambronero et al. [2] note, choosing a DSL requires a balance between search tractability and expressiveness.

Regardless of how narrowly a DSL ultimately shapes the search space, what the search is looking for is determined by the type of specification used to describe a task, and it is this form that sets PBE apart from the other PS paradigms.

3.2 Programming by Example

PBE is the special case of PS in which the specification consists of a set of input-output examples rather than a formal condition or a natural language description [10, 16]. Formally, Devlin et al. [5] formalize the goal as finding, from a specification comprising a set of pairs $(I_1, O_1), \dots, (I_n, O_n)$, a program P such that $P(I_j) = O_j$ for every pair j . The doubling-and-sort task from Chapter 1 is one such specification: a pair like $([3, 1, 2], [2, 4, 6])$ tells a PBE approach nothing about how to double and sort a list in general, only that this particular input must map to this particular output, and it is left to the search to find a program consistent with pairs like it. Because a PBE user typically cannot write or read the program that is being synthesized, PBE lowers the barrier to automating a task to providing a handful of examples of the desired behavior [14, 9].

Underdetermination. A small number of examples rarely pins down a single program: several different programs can satisfy the same examples while behaving differently on inputs that were not shown. Table 3.2 illustrates this with a task that maps a list of numbers to a single number.

Table 3.2: Two candidate programs, “return the maximum” and “return the last element,” both satisfy Examples 1 and 2. Only Example 3 disambiguates between them.

	Input	Output
Example 1	[5, 2, 9]	9
Example 2	[1, 4]	4
Example 3	[7, 3, 6]	6

Given only Examples 1 and 2, both “return the maximum element” and “return the last element” are consistent programs, since the maximum happens to be the last element in both cases. Example 3 breaks this coincidence: the maximum of $[7, 3, 6]$ is 7, but the expected output is 6, the last element. Only the second program generalizes correctly to this new input. This is what is meant by underdetermination in PBE: because examples are an incomplete specification, satisfying the given examples is not sufficient to guarantee that a program has captured the intended transformation [10, 26]. In the extreme case, a program need not even look at the input: if the expected outputs of the given examples happen to coincide, a program that always returns the same constant can already satisfy them without encoding any rule about the input at all.

Underdetermination is also why PBE requires more than matching the given examples verbatim. A PBE approach must reason about which of the potentially many programs consistent with the specification is the one the user actually intended, for instance by preferring a simpler or more general program over one that merely memorizes the given examples. Providing more input-output examples per task reduces, but does not eliminate, this ambiguity: Wang et al. [26] show that it is generally impossible to guarantee that a program will behave correctly on every unseen input from a finite set of examples alone. Consequently, evaluating a PBE approach also requires distinguishing between examples used to specify a task and examples used to test whether the synthesized program actually generalizes.

Two Levels of Splitting. This distinction applies at two different levels. At the level of an individual task, its input-output examples are split into a specification part, given to the synthesis approach as the task description, and a held-out test part, used only to check whether the resulting program produces the correct output on inputs it did not see during synthesis. At the level of an entire dataset, the set of available tasks is itself split into a portion used to develop or train the approach, for example to fit a neural component that guides the search, and a portion of held-out test tasks that the approach has not encountered before and that is used to evaluate its performance. Both levels of splitting are necessary, and they check different things. The task-level split asks whether an individual solution generalizes beyond its own specification. The dataset-level split asks whether the approach as a whole generalizes to tasks it has not been developed on. Both levels presuppose that it can be determined exactly when a program solves a task.

3.3 Accuracy as an Evaluation Metric

Accuracy is the standard metric for evaluating classification-style predictions and is defined generally as the proportion of correctly predicted examples relative to the total number of examples [20]. This general definition is commonly instantiated for PS and PBE by treating the exact match between a generated and an expected output as the correctness criterion: a task counts as solved only if the generated output matches the expected output on every one of the task’s test examples, not merely on some of them [3, 16]. Over a set of tasks, accuracy is then the proportion of tasks solved in this sense:

$$\text{accuracy} = \frac{\text{number of correctly solved tasks}}{\text{number of all tasks}}.$$

A value of 1 means every task was solved, and a value of 0 means none was. Accuracy owes its popularity to being easy to understand, cheap to compute, and directly comparable across approaches, particularly in benchmarks where each task has a single, unambiguous expected output [20].

In PBE specifically, however, this complete-match requirement has the same consequence already visible in Chapter 1: accuracy is inherently binary at the level of a single task, either fully solved or not, giving a clear verdict on complete correctness but next to no information about the quality of a partially correct solution.

A Running Example Task. To make the following chapters concrete, this thesis returns to the doubling-and-sort task from Chapter 1 and uses it as one running example throughout: transforming a list of integers by doubling each element and sorting the result. Table 3.3 formalizes this task through three input-output examples, together with a reference program that solves it.

Table 3.3: The running example task used throughout this thesis: doubling and sorting a list of integers.

Input	Output
[3, 1, 2]	[2, 4, 6]
[5, -1, 0]	[-2, 0, 10]
[4, -3, 1]	[-6, 2, 8]

Reference program: $\text{Map}(\times 2)(\text{Sort}(\text{input}))$

Under the definition given above, this reference program achieves an accuracy of 1 on the task in Table 3.3, since it reproduces the expected output on every example. Now consider a second program, $\text{Map}(+2)(\text{Sort}(\text{input}))$, which sorts the input correctly but adds 2 to each element instead of doubling it. This program produces [3, 4, 5] instead of [2, 4, 6] for the first example, so it fails on every test example and receives an accuracy of 0, the same score as a program with no discernible relation to the task at all. Yet the two programs are not equally wrong. The second one correctly identifies that the input must be sorted and applies an operation to each element; it deviates from the reference program in only one respect. Accuracy, being binary by construction, cannot express this difference. A program that noticeably comes closer to the reference than an arbitrary wrong one receives the same value of 0 as that program.

A program that comes close to the reference, however, clearly deserves more than a zero. This calls for a yardstick that does not stop at the output but looks at the program

itself: its structure, its building blocks, their arrangement. What such a yardstick can look like is the subject of the next chapter.

4 Methodology

Answering the research question requires a concrete methodology, developed here in three parts. Section 4.1 resolves a problem raised at the end of Chapter 2: DeepCoder’s fixed DSL and DreamCoder’s evolving lambda-calculus library share no common vocabulary, so their programs must first be brought onto comparable ground before anything about them can be compared. The four token-based metrics this thesis contributes follow in Section 4.2. Section 4.3 closes the chapter by explaining how these metrics are read jointly, together with the BSS from Chapter 3, to answer the research question. Chapter 5 then fixes the concrete experimental conditions under which DeepCoder and DreamCoder are evaluated.

4.1 Data Conversion

DeepCoder’s programs are already sequences of DSL operation calls, so tokenizing them is direct: each operation, together with its arguments, becomes one token. DreamCoder’s programs, by contrast, are expressions of a typed lambda calculus and can express the same operation in many different but semantically equivalent syntactic forms. A direct, token-by-token comparison between a DeepCoder reference program and a DreamCoder solution would therefore be meaningless even when both compute the same transformation. Identical behavior simply would not translate into identical tokens.

To close this gap, DreamCoder’s lambda-calculus expressions are mapped onto the same abstract operation vocabulary already used for DeepCoder, independently of how a given operation happens to be nested or indexed within the expression. A lambda-calculus subexpression that reverses a list, for instance, is mapped to the same token that DeepCoder’s DSL uses for its built-in reverse operation, regardless of the exact variable binding used to express it. This mapping operates at the level of operation *families* rather than exact syntax: it recognizes that two expressions perform the same kind of operation even when one spells it out more explicitly than the other. A representative case from the evaluation illustrates this: a reference program consisting of a

TAKE and a REVERSE operation is matched against a DreamCoder solution that reaches the same result differently, by first concatenating the input onto an empty list and only then reversing the concatenated result, a functionally equivalent but syntactically longer detour through three operation tokens instead of one. After mapping, both programs are expressed in the same shared vocabulary and can be compared using the token-based metrics defined next, even though neither their length nor their syntactic form agrees.

This mapping is necessarily incomplete. It only covers operation families for which a correspondence was defined; an operation expressed in a way that was not anticipated simply remains unmapped. This limitation, and how it may have affected specific results, is revisited in Chapter 6. Despite these gaps, most program pairs share a common token vocabulary after mapping.

4.2 Token-based Program Metrics

This thesis introduces four such metrics, each isolating a different way a generated program can deviate from the reference: the Program Operation Score (POS), the Program Position Score (PPS), the Program Sequence Score (PSS), and the Program Edit Score (PES). A single aggregate similarity score, such as BLEU (Section 2.2), could not distinguish an approach that finds the right operations but combines them in the wrong order from one that fails to find the right operations at all, even though both could produce the same aggregate value; keeping presence, position, contiguous structure, and overall edit effort as four separate numbers instead makes it possible to attribute a low score to a specific kind of error. By convention, if both the reference and the generated token sequence are empty, every metric below evaluates to 0.

The four metrics are illustrated throughout using one example: the task of doubling every element of a list and then sorting the result, formalized in Table 4.1 through three input-output examples together with a solving reference program.

Table 4.1: The example used throughout this chapter: doubling and sorting a list of integers.

Input	Output
[3, 1, 2]	[2, 4, 6]
[5, -1, 0]	[-2, 0, 10]
[4, -3, 1]	[-6, 2, 8]

Reference program: `Map( $\times 2$ )(Sort(input))`

Its reference program tokenizes as follows:

$$p = \text{Map}(\times 2)(\text{Sort}(\text{input})) \quad \longrightarrow \quad p = [\text{Map}, *2, \text{Sort}, \text{INPUT}]$$

Alongside it, consider a third candidate program that both reorders the two operations and substitutes one of them:

$$\hat{p} = \text{Sort}(\text{Map}(+2)(\text{input})) \quad \longrightarrow \quad \hat{p} = [\text{Sort}, \text{Map}, +2, \text{INPUT}]$$

Unlike the near-miss program from Section 3.3, which only replaced one token, this candidate also changes the order in which the operations appear, which is what lets the four metrics below disagree with one another instead of all telling the same story.

4.2.1 Program Operation Score

The POS asks a narrow question first: are the right building blocks present at all, regardless of where they end up? It treats the two token sequences as multisets instead of ordered sequences, so neither order nor position affects the result. A plain set-based check would already be satisfied by a single occurrence of a required operation; since PBE programs frequently reuse the same operation with different arguments, for instance in chained `Map` calls, a multiset comparison is used instead, so that an operation must occur the correct number of times, not merely at all.

$$\text{POS}(p, \hat{p}) = \frac{\sum_{t \in T} \min(\text{count}_p(t), \text{count}_{\hat{p}}(t))}{\max(|p|, |\hat{p}|)}$$

Here T is the set of all tokens occurring in p or $\hat{p}$, and $\text{count}_p(t)$ counts how often token t occurs in p . For the running example, the tokens `Map`, `Sort`, and `INPUT` occur in both p and $\hat{p}$; only `*2` does not, since $\hat{p}$ uses `+2` in its place. $\text{POS}(p, \hat{p})$ therefore comes out to $3/4 = 0.75$: three of the four reference tokens turn up somewhere in the generated program, even though its order differs entirely and one operation was substituted. Where these three tokens sit in the generated program plays no role in this number, a blind spot that would score a candidate with completely swapped operations just as highly as one that hits them in the right place.

4.2.2 Program Position Score

Index by index, the PPS compares both sequences without any alignment step: it does not first align the two sequences, the way the PES does below via edit-distance alignment, and only then measure how far tokens drifted. An alignment-tolerant variant would absorb exactly the position sensitivity this metric is meant to expose, making it largely redundant with the PES; the raw, unaligned comparison instead penalizes any positional shift, however small, so that ordering problems stay visible.

$$\text{PPS}(p, \hat{p}) = \frac{\sum_{i=1}^{\min(|p|, |\hat{p}|)} \mathbf{1}(p_i = \hat{p}_i)}{\max(|p|, |\hat{p}|)}$$

where $\mathbf{1}(p_i = \hat{p}_i)$ is 1 if the tokens at position i match and 0 otherwise. In the running example, only the last position, `INPUT`, matches between p and $\hat{p}$; every earlier token shifted because the two operations were reordered. $\text{PPS}(p, \hat{p})$ comes out to only $1/4 = 0.25$, far below the POS of 0.75 for the same pair of programs, even though three of the four tokens are present somewhere in $\hat{p}$. What the PPS makes visible here is exactly the reordering that the POS, by design, cannot see. Whether the shifted tokens at least remain together as a contiguous block, however, is not something position alone can determine, since it penalizes every shift independently, regardless of whether neighboring tokens move together or are scattered individually across the sequence.

4.2.3 Program Sequence Score

This thesis uses a strict variant of the LCS-based comparison, measuring the longest *contiguous* common token block rather than the classical, gapped LCS. A gapped LCS would count two tokens as order-preserving even with several unrelated tokens between them, a signal the POS already captures; requiring contiguity instead isolates whether operations that sit directly next to each other in the reference program, and therefore typically implement one sub-transformation together, remain adjacent in the generated program.

$$\text{PSS}(p, \hat{p}) = \frac{\text{LCB}(p, \hat{p})}{\max(|p|, |\hat{p}|)}$$

where $\text{LCB}(p, \hat{p})$ is the length of the longest contiguous block common to both sequences. Reordering the two operations in the running example leaves no contiguous pair of tokens intact between p and $\hat{p}$; the longest common block is a single token (`Map`, `Sort`, or `INPUT`, each individually). $\text{PSS}(p, \hat{p})$ works out to $1/4 = 0.25$ as a result. The

same reordering that the POS could not see, and that the PPS penalized at every shifted position, also destroys the one contiguous two-token block, `Sort(input)`, that the reference program contains. POS, PPS, and PSS thus provide three separate values for three different kinds of deviation, presence, position, and contiguity, but leave open how much total effort lies behind a candidate; condensing these three signals into a single aggregated figure is the task of the last of the four metrics.

4.2.4 Program Edit Score

Presence, position, and contiguity are condensed by the PES into a single number of required changes, using the Levenshtein distance. Plain Levenshtein distance, allowing insertion, deletion, and substitution, is used rather than a variant such as Damerau-Levenshtein that additionally allows transposing adjacent tokens: a transposition is already reflected in the PPS and PSS above, so adding it here would make the PES partly redundant with those two metrics instead of contributing an independent, aggregate signal.

$$\text{PES}(p, \hat{p}) = 1 - \frac{\text{EditDistance}(p, \hat{p})}{\max(|p|, |\hat{p}|)}$$

A value of 1 means the two programs are token-for-token identical; lower values mean more edits are required to turn one into the other. Transforming $\hat{p}$ into p in the running example takes three edits: substituting each of the first three tokens in turn, `Sort` $\rightarrow$ `Map`, `Map` $\rightarrow$ `*2`, and `+2` $\rightarrow$ `Sort`, already reaches p while leaving `INPUT` untouched. That puts $\text{EditDistance}(p, \hat{p})$ at 3, so $\text{PES}(p, \hat{p}) = 1 - 3/4 = 0.25$. That this value coincides with the PPS and PSS for this particular pair of programs is not a general pattern; it reflects that reordering two adjacent tokens happens to cost as many edits here as it costs matched positions and contiguous blocks. The POS, at 0.75, remains the outlier of the four: it is the only one of the four metrics blind to the reordering itself, which is precisely what lets it show that the right operations were nonetheless found.

With POS, PPS, PSS, and PES, four numbers are therefore available for each pair of programs, each expressing a different kind of structural deviation, from pure presence to aggregated edit effort. Four separate numbers are not yet an interpretation, however: what a high POS combined with a low PPS actually says about a candidate, and how these four values relate to the BSS that alone decides whether a task is solved or not, only becomes clear once they are read together.

4.3 Comparison and Interpretation of the Metrics

Before the rest of this section turns to how the four metrics are read together, Table 4.2 recaps what each one captures and the value it takes on the running example from Section 4.2.

Table 4.2: The four token-based metrics, what each one captures, and its value on the running example from Section 4.2.

Metric	What it captures	Running example
POS	Presence of the right tokens (multiset), ignoring order and position	0.75
PPS	Whether tokens occur at exactly the same position	0.25
PSS	Length of the longest contiguous common token block	0.25
PES	Overall number of token edits needed to match the reference	0.25

The four metrics are not intended to be read in isolation, but jointly, since each captures a different way a generated program can diverge from the reference. A high POS together with a low PPS points to correct tokens that were simply placed incorrectly, an ordering problem rather than a missing capability. A low POS, by contrast, means components of the reference program are missing or were replaced outright. Contiguous parts of the reference structure survive as a unit when the PSS is high; when it is low, at best isolated tokens match without forming any shared substructure. The PES complements this by quantifying the total edit effort behind whatever pattern the other three metrics reveal.

The BSS remains informative at the output level: it shows whether the final output is correct. The four program-level metrics add a complementary layer by showing why an output might be wrong, or why it might be right for reasons the BSS cannot distinguish. Because several different programs can produce the same output, a generated program can be judged correct at the output level while still deviating structurally from the reference; in such a case, the BSS reaches its maximum value while the token-based metrics can still be comparatively low: the correct output, it turns out, was produced by a structurally different program. Conversely, a generated program can be judged entirely wrong at the output level while still scoring highly on individual token-based metrics. Substantial parts of the reference structure were correctly recognized here, even though the final output was not. Reading the BSS together with POS, PPS, PSS, and PES therefore enables a more differentiated interpretation of a result than either could provide alone.

This comparison has a limit worth stating explicitly: none of the four metrics constitutes a semantic equivalence check. They measure structural closeness to one specific reference program, not whether two programs compute the same function for all possible inputs; two structurally very different programs can still be functionally identical, and the metrics have no way to detect this beyond what the BSS already shows on the given test examples. The asymmetry also runs the other way: a single substituted token, as in the near-miss program from Section 3.3, already changes the computed function on most inputs while barely moving any of the four metrics, so a high score is no more a guarantee of semantic correctness than a low score is a guarantee of semantic difference. What the joint reading does reveal is whether a structural deviation from the reference coincided with a correct or an incorrect output on the tasks actually tested.

This settles the tools with which DeepCoder and DreamCoder are compared for the remainder of this thesis. How this comparison unfolds in detail, under what conditions both approaches were run, and how their results are aggregated is the subject of the next chapter.

5 Evaluation

Before any result can be reported, four things have to be fixed: the task domain both approaches are evaluated on, the specific configurations compared within each approach, the hyperparameters and reproducibility guarantees behind every number, and the convention used to aggregate and report them. This chapter fixes each in turn, in that order.

5.1 Task Domain

The list processing domain concerns the transformation of integer arrays. Its programs are built from a fixed set of 34 primitives, taken from the DSL introduced by Balog et al. [1]. Seven of these operate directly on a list without any further argument: **HEAD** and **LAST** extract the first and last element, **MINIMUM** and **MAXIMUM** the smallest and largest, **REVERSE** reverses the element order, **SORT** sorts the list in non-decreasing order, and **SUM** adds up its elements. Three further operations combine a list with an integer argument: **TAKE** and **DROP** keep or discard a given number of leading elements, and **ACCESS** returns the element at a given index. The remaining five operations are higher-order: **MAP** and **FILTER** apply a unary function to every element or keep only the elements satisfying a unary predicate, **COUNT** counts how many elements satisfy a predicate, **SCANL1** computes a running left-fold over the list using a binary function, and **ZIPWITH** combines two lists element-wise using a binary function, truncating to the shorter of the two. Each of these five higher-order operations is paired with one of 19 lambda functions to form a complete program step: ten unary arithmetic functions (for example doubling, halving, or negating a value), four unary predicates (for example testing whether a value is positive or even), and five binary functions (addition, subtraction, multiplication, minimum, and maximum). Fifteen base operations together with 19 lambdas add up to the full set of 34 primitives.

A program is written as a sequence of these operation calls, each one creating a new variable from the task’s inputs or from variables computed by earlier calls; the value of

the last variable is the program’s output. Program length counts these operation calls rather than raw tokens: a lambda function paired with a higher-order operation, such as multiplying every element by -1 inside a `MAP` call, is counted as a single operation together with that call, not as two separate steps. The programs used throughout this thesis are restricted to a length of at most two such operations.

Table 5.1 illustrates this on a real task from the underlying dataset. Its reference program first negates every element of the input list, then returns the last element of the result, equivalently and more simply, it returns the negation of the input’s last element.

Table 5.1: A list processing task with five input-output examples, together with a reference program of length two that satisfies all of them.

	Input	Output
Example 1	<code>[-205, -137, 172, 94, 179, 143, 144, -15, 80, 195, 196, 146]</code>	<code>-146</code>
Example 2	<code>[149, 100, -247, -215, 114]</code>	<code>-114</code>
Example 3	<code>[-193, 77, 212, -206, 134, 113, -66, 251, -85, -229, 157, -14]</code>	<code>14</code>
Example 4	<code>[120, -119, -247, 236, -102, 210, 196, 248, 84, 172, 225]</code>	<code>-225</code>
Example 5	<code>[-74, -229, 182, -199, 37, 210, 166, -151, -89]</code>	<code>89</code>

Reference program: `LIST|MAP,*-1,0|TAIL,1`

Task inputs vary between one and three such arguments, each either a single integer or a list of up to 20 integers, with all integer values, list elements included, ranging between -256 and 256 . Input values are not sampled uniformly at random; the permissible range for each argument is first propagated backward from the operations the reference program applies to it, so that every sampled input keeps the resulting computation well-defined and within this range. The corresponding output is then obtained simply by running the reference program on the sampled input. Each task is specified by five such input-output examples, and a task counts as solved once a search finds a program consistent with all five within the available search budget. For the length restriction used in this thesis, 100 test tasks and 2,365 training tasks are available; the training portion is used to fit each approach’s learned component, and all reported metrics are computed on the held-out test portion only.

DreamCoder represents programs differently from this fixed DSL: its programs are expressions of a typed lambda calculus, and instead of a fixed vocabulary, it grows its own library of reusable components over repeated training cycles. Its programs are therefore first mapped onto the same 34-primitive vocabulary described above before

they can be evaluated by the same metrics. After this mapping, 99 of the 100 test tasks remain usable, since a small number of expressions cannot be matched to any of the 34 primitives; all three DreamCoder configurations examined in this thesis are trained on a 500-task subset of the corresponding training split.

Within this shared domain, however, DeepCoder and DreamCoder do not compete against each other as single, fixed approaches: each is run in several configurations that specifically isolate individual mechanisms, in order to clarify which part of an approach is actually responsible for a result.

5.2 Compared Configurations

The base code for both approaches builds on the publicly available implementations of DeepCoder [12] and DreamCoder [8]. The extensions required for this thesis, the mapping between DreamCoder’s expressions and the DSL vocabulary above, the four token-based metrics this thesis contributes, and a mechanism for tracking partial candidates on unsolved tasks, were implemented on top of these bases and are published at [28] and [29], respectively.

For DeepCoder, the search itself is evaluated in two configurations that isolate the contribution of its neural component: a plain depth-first search over the 34 primitives, tried in a fixed default order, and the same search guided instead by a neural attribute predictor after it has been trained on the training split to estimate which of the 34 primitives a solving program is likely to contain. Comparing these two configurations at the same fixed search budget separates what an unguided search can already find on its own from what the neural guidance adds on top of it.

For DreamCoder, its two learning mechanisms are evaluated both individually and combined, giving three configurations per run. Consolidation grows the library of reusable components by identifying and storing subexpressions that compress previously found solutions, allowing later tasks to be solved with fewer, higher-level steps. Recognition instead trains a neural model that proposes which candidate programs are likely to solve a given task, guiding the search toward promising regions of the current library without changing the library itself. The three configurations, Consolidation without Recognition, Recognition without Consolidation, and both together, are each run for the same fixed number of training cycles on the same training tasks, so that any performance difference between them can be attributed to the specific mechanism rather than to unequal training conditions.

This settles five named configurations, but their names alone do not yet determine whether a measured difference between them actually reflects the respective mechanism or merely random fluctuation: for that, the search budget, training duration, and handling of randomness behind every individual number must also be fixed.

5.3 Hyperparameters and Reproducibility

Unless stated otherwise, the DeepCoder results refer to a search budget of $gas = 1,000$ candidate programs per task. The DreamCoder configurations are each trained for 3 wake-sleep cycles with an abstraction arity of 3, meaning that a newly learned library component may combine up to three existing pieces at once.

For DeepCoder, the plain, untrained configuration is fully deterministic: its depth-first search contains no learned or randomized component, so repeated runs are byte-identical regardless of seed. The trained configuration depends on the attribute predictor’s weight initialization and training-batch order, both seed-dependent; it is trained for 10 epochs and reported individually for each of five seeds (0–4), alongside the mean and population standard deviation across them, both in the result tables and in the accompanying bar charts. DreamCoder exposes a random-seed parameter as well, but it only affects a task-batching strategy that this thesis does not use and has no effect on library learning or the recognition model under the configuration used here; each DreamCoder configuration is therefore reported as a single run rather than an aggregate over several. DreamCoder’s enumeration and testing timeout are each set to 15 seconds per task.

Five seed-dependent DeepCoder runs and a single DreamCoder run per configuration, however, do not by themselves yield a single reportable number: in particular for tasks on which a search fails to find a solving program, it must first be settled whether and how they factor into a mean at all.

5.4 Reporting Convention

For every metric, both the mean over *all* test tasks and the mean over only the *solved* tasks are reported. Unsolved tasks enter the token-based metrics with the value 0 by convention, regardless of whether an incomplete candidate was captured during the search; reporting only the all-task mean would therefore understate how structurally close the search actually gets on the tasks it does solve, regardless of how many tasks remain unsolved overall.

Domain, configurations, hyperparameters, and aggregation convention together thus determine which number may be reported at all for which run. What these numbers actually show for DeepCoder and DreamCoder, once they are collected under these conditions, is the subject of the next chapter.

6 Results and Discussion

This thesis asks what additional insight into the performance of PBE approaches evaluation metrics beyond a BSS can provide, and derives three expectations from that question: a BSS hides aspects of search behavior that only become visible at the program level; analyzing partial correctness yields additional information, even for tasks a BSS counts as unsolved; and program-level metrics can, in specific cases, explain more informatively why a task was or was not solved than a BSS alone. This chapter tests all three. Performance and program quality are first reported in two separate sections before being related to each other: the solve rate as a pure performance measure, then the four token-based metrics as a measure of the quality of the underlying programs; the two are then brought together to test how far the resulting pattern holds, before the chapter closes with what the evaluation cannot show.

6.1 Performance

Table 6.1 reports the solve rate, the BSS averaged over a set of tasks, for each configuration examined.

Table 6.1: Solve rate (BSS) across all five configurations.

Configuration	Tasks	Solved	Solve rate
DeepCoder, without training	100	53	53.0%
DeepCoder, with training, mean $\pm$ std	100	67.8	67.8% $\pm$ 1.8%
DreamCoder, Consolidation	99	25	25.3%
DreamCoder, Recognition	99	31	31.3%
DreamCoder, combined	99	34	34.3%

For DeepCoder, training the attribute predictor raises the solve rate substantially at the same search budget, from 53.0% to a mean of 67.8% $\pm$ 1.8% across five seeds, with individual seeds ranging from 66 to 70 solved tasks out of 100. For DreamCoder, the

combined configuration reaches the highest solve rate at 34.3%, ahead of Recognition alone (31.3%) and Consolidation alone (25.3%); since all three configurations share the same training tasks and iteration count, this difference is attributable to the two mechanisms rather than to unequal training conditions. If one of the two mechanisms were redundant with the other, solving only tasks the other already solves anyway, the combined configuration could not exceed the better of the two individual configurations, since it would then merely cover the same tasks twice. That the combination instead exceeds both Recognition alone and Consolidation alone is only possible if each mechanism solves tasks that remain closed to the other on its own, evidence that Consolidation and Recognition address different parts of the search problem rather than one making the other redundant.

The solve rate alone, however, says nothing about how structurally close a generated program is to its reference.

6.2 Quality of the Generated Programs

Even among solved tasks, not every generated program is the reference program itself. The fraction of solved tasks whose program is token-for-token identical to the reference, equivalently the fraction reaching a PES of 1, is comparatively small: for DeepCoder, only 11.0% of all tasks (20.8% of the solved ones) are matched exactly without training, rising to a mean of $21.0\% \pm 1.3\%$ of all tasks ($30.9\% \pm 1.1\%$ of the solved ones) with training. In the large majority of solved cases, the search therefore finds an alternative program instead, one that is functionally correct but structurally different from the reference, precisely the distinction accuracy alone cannot draw.

What the two training procedures actually reward, rather than any weakness in the search itself, explains why so many solved tasks nonetheless fall short of a PES of 1. DeepCoder’s attribute function maps a program onto a binary vector that only records whether a function occurs anywhere, not in what order or nesting; the network consequently learns to predict operations, never an arrangement. The subsequent depth-first search does try these predicted functions in descending order of probability, but among the example-satisfying compositions it finds, it does not favor one that resembles any particular reference structure, since no such reference is ever known to the training process. Every candidate program that reproduces the same examples counts equally for training. This is why a solved program regularly contains the right building blocks without arranging them in the reference’s order: the training objective itself does not

distinguish between one valid arrangement and another as long as both produce the same output.

For DreamCoder, the same effect sits even deeper inside the training loop itself. Consolidation extends the library according to a prior $P[L]$ that explicitly favors compact libraries, and adopts as new primitives precisely those sub-expressions that most strongly compress the corpus of tasks solved so far, regardless of how closely the resulting solution matches the decomposition used by any one reference program. Recognition, in turn, concentrates its probability mass on a single canonical form whenever several syntactically different but semantically equivalent programs solve a task, but that canonical form is whichever one the current library itself makes most likely, not the external reference from the evaluation dataset, which the model never sees during training, since its training pairs come exclusively from its own solved tasks and self-sampled fantasies. A training objective that actively searches for the most compact representation across tasks has no reason to move toward a task-specific reference structure, and moves systematically away from it whenever a more compact, cross-task-reusable alternative exists.

For the metrics developed in this thesis, this means they consistently measure something whose optimization neither training procedure ever rewards. A PES below 1 on a solved task is therefore the expected consequence of a training objective that rewards only example correctness and assigns no value anywhere to structural closeness to a specific reference, not evidence that the underlying search worked incorrectly. Since no optimization pressure pushes in this direction, the four token-based metrics supply information here that neither the BSS nor the training signal itself already contains. They are reported individually below, beginning with the least strict of the four.

6.2.1 Program Operation Score

Table 6.2: POS, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.313	0.591
DeepCoder, with training, mean $\pm$ std	0.441 ± 0.016	0.650 ± 0.007
DreamCoder, Consolidation	0.064	0.253
DreamCoder, Recognition	0.085	0.272
DreamCoder, combined	0.079	0.230

DeepCoder’s POS (Table 6.2) is clearly higher than DreamCoder’s in every configuration. Part of this follows directly from the normalization pipeline that maps both approaches’ programs onto a shared vocabulary: mapping DreamCoder’s lambda-calculus solutions onto abstract operation families tends to produce shorter, coarser token sequences than DeepCoder’s directly tokenized DSL programs, which shrinks the multiset overlap the POS can find even when the underlying solution is reasonable. This coarseness is not an inconsistency in the tokenization but a deliberate choice built into the mapping itself: different syntactic variants of the same operation in DreamCoder’s library must be mapped onto the same family for a comparison with DeepCoder’s DSL to be possible at all, and this abstraction holds regardless of how closely the tokenization otherwise matches in detail on both sides.

The POS treats both token sequences as multisets and completely ignores where a token sits; a candidate with fully swapped operations would score the same as one that places every token correctly.

6.2.2 Program Position Score

Table 6.3: PPS, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.292	0.551
DeepCoder, with training, mean $\pm$ std	0.437 ± 0.017	0.645 ± 0.008
DreamCoder, Consolidation	0.028	0.110
DreamCoder, Recognition	0.026	0.083
DreamCoder, combined	0.049	0.142

Every configuration scores lower on the PPS (Table 6.3) than on the POS reported above, confirming that the tokens found are not always where the reference program places them. The drop is much steeper for DreamCoder: its PPS never exceeds 0.142, far below its own POS, while DeepCoder’s PPS stays close to its POS. This asymmetry is not simply a consequence of DreamCoder’s library changing over the course of a run; even with a fixed library, there would be no reason why a token’s position in a normalized lambda-calculus expression should match its position in a sequentially composed DeepCoder DSL program. DeepCoder’s DSL forces an operation into a specific place in the call sequence, fixed by the data dependencies between steps; DreamCoder’s

lambda-calculus expressions have no analogous sequential structure, and their position after normalization is an artifact of how the expression tree is serialized, not of the functional order of the computed steps. This forced ordering also explains the other side of the asymmetry: once DeepCoder finds the right operations, the data dependencies between steps often allow only a single valid arrangement, or very few, which is why its PPS stays so close to its POS. That the library also grows over the course of a run compounds this problem further: the same operation can be expressed through different, currently active library components at different points in time, which guarantees no stable position even within a solution built from the right operation families.

Whether a low PPS value reflects a fully scattered solution or one whose tokens were merely shifted as a block is something the PPS alone cannot distinguish, since it penalizes every shift independently, regardless of whether neighboring tokens move together or are scattered individually across the sequence. The PSS measures exactly that: how long the largest contiguous block is that both programs share.

6.2.3 Program Sequence Score

Table 6.4: PSS, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.282	0.532
DeepCoder, with training, mean $\pm$ std	0.418 ± 0.018	0.616 ± 0.011
DreamCoder, Consolidation	0.054	0.213
DreamCoder, Recognition	0.072	0.228
DreamCoder, combined	0.071	0.206

The PSS (Table 6.4) stays clearly below the POS for both approaches, but relates to the PPS differently. For DeepCoder, it sits close to, even slightly below, its own PPS (0.616 versus 0.645 among solved tasks): wherever an operation is found, it tends to land directly in the right position, without the detour of a merely shifted block. For DreamCoder, by contrast, the PSS is clearly higher than the PPS in every configuration, between 45% and 175% higher among solved tasks. Since the PSS only rewards tokens staying adjacent, while the PPS penalizes even the smallest shift independently, this gap shows that DreamCoder’s solutions more often share a contiguous sub-block with the reference that is merely shifted to a different position, rather than being fully scattered,

a pattern the PPS alone cannot make visible.

Relative to the POS, the PSS is also considerably more stable than the PPS. For DreamCoder, the PSS reaches 84 to 90% of its own POS among solved tasks throughout: 0.213 out of 0.253 for Consolidation, 0.228 out of 0.272 for Recognition, 0.206 out of 0.230 for combined, a share close to what DeepCoder also reaches (90 to 95%). The PPS’s share of the POS, by contrast, fluctuates for DreamCoder only between 31% and 62%, markedly lower and far less consistent across the three configurations. Even when the search finds the right operations, then, how many of them land exactly in the right position remains uncertain for DreamCoder, yet a large, largely stable share of them at least form a contiguous block with the reference somewhere. This is because tree serialization tends to keep semantically related sub-expressions together as a block, for instance an operation together with the operation supplying its argument, even as where in the overall sequence that block ends up shifts; a growing library or a different serialization order therefore tends to shift the block as a whole rather than tearing it apart internally. The PSS is accordingly the more robust of the two indicators against the representation-driven positional shift that causes the PPS to collapse so sharply for DreamCoder.

POS, PPS, and PSS thus each illuminate a separate facet of the same program pairs, yet none of them sums up how much total editing lies between the generated and the expected program; the last of the four metrics provides that overall view.

6.2.4 Program Edit Score

Table 6.5: PES, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.292	0.551
DeepCoder, with training, mean $\pm$ std	0.437 ± 0.017	0.645 ± 0.008
DreamCoder, Consolidation	0.033	0.130
DreamCoder, Recognition	0.045	0.142
DreamCoder, combined	0.064	0.186

For DreamCoder, the PES (Table 6.5) lies between the PPS and PSS in every configuration, not close to either one: an edit sequence can compensate for a shifted but intact block through a few insertions and deletions, and so credits part of the pattern the PSS

fully captures and the PPS completely misses, though not to the same degree as the PSS itself. For DeepCoder specifically, the PES matches the PPS almost exactly in every configuration in this table, not merely on the same order of magnitude. This closeness follows from program length: DeepCoder’s reference and generated programs are close enough in length that the cheapest way to turn one into the other is usually a straight sequence of position-wise substitutions, the same operation the PPS penalizes, so the two metrics measure nearly the same thing on this particular dataset. This convergence is specific to DeepCoder’s short, similarly sized programs and should not be expected to hold in general, one reason this thesis keeps the PPS and PES as separate metrics rather than assuming one could stand in for the other.

As an aggregate measure, the PES can also be read directly as editing effort. For DeepCoder’s trained configuration, a PES of 0.645 among solved tasks means that, on average, a good third of the tokens would already need to change to get from the generated to the reference program, even among the tasks counted as fully solved. For DreamCoder’s combined configuration, this share exceeds four-fifths (a PES of 0.186): a task solved under the BSS is thus, on average, farther from its reference than a pure presence or continuity view alone would suggest.

Within the three DreamCoder configurations, the PES also shows a pattern opposite to the PSS’s: while the combined configuration reaches the lowest PSS of the three among solved tasks, it reaches the highest PES, 0.186 against 0.142 for Recognition and 0.130 for Consolidation. This traces back to its markedly higher PPS, 0.142 against 0.083 and 0.110. This higher PPS value, however, can no longer be attributed to the search alone: since each of the three configurations actually solves a different subset of the 99 tasks, a solved-only mean compared across the three configurations does not compare the same set of tasks, only whichever tasks each configuration manages to solve at all; how much of combined’s higher PPS reflects the additionally solved tasks themselves and how much reflects genuinely more consistent positioning cannot be separated with the data available here. Because the PES lies between the PPS and PSS, this unusually high positional accuracy nonetheless pulls the aggregate value upward, even though continuity is weaker than in the other two configurations.

With all four metrics reported individually, the question now becomes whether they agree in their ranking, and how that ranking relates to the performance reported at the outset.

6.3 Connection between Performance and Program Quality

A first hint that performance and program quality can diverge comes from DreamCoder’s combined configuration. Although it reaches the highest solve rate among the three DreamCoder configurations (Table 6.1), its PSS among solved tasks (0.206) is the lowest of the three, not the highest, a first, concrete instance of the disagreement between the BSS and the structural metrics that this thesis set out to make visible. This follows from the same mechanism that gives it the higher solve rate. In the combined configuration, Consolidation and Recognition grow together on the same library: new, compressed building blocks shorten the search space, while the Recognition model increasingly steers the search toward this changing library rather than toward the original, smaller-grained primitives. Tasks that become additionally solvable this way, compared to Consolidation or Recognition alone, tend to be exactly those reachable only through an already repeatedly compressed, cross-task building block, not through directly recomposing the same basic operations the reference program also uses. The combination’s additional solving power therefore comes disproportionately from precisely the tasks whose found solution is structurally farthest from the reference, which pulls down the PSS mean over this configuration’s solved tasks even though its solve rate is the highest of the three configurations.

6.3.1 The POS Advantage: A Recurring Pattern

One pattern holds across both approaches and every configuration examined: the POS is the highest of the four metrics among solved tasks, without exception, in Tables 6.2–6.5. Both DeepCoder and DreamCoder find the right operations more often than they combine them correctly. Figure 6.1 summarizes the solve rate together with all four token-based metrics for DeepCoder; Figure 6.2 does the same for DreamCoder’s three configurations.

Of the four metrics, the POS tests the least strict condition: whether an operation occurs at all, regardless of where or in what order. That POS leads the other three metrics in every single configuration follows already from exactly this weaker condition and the definitions of the four metrics, and holds for every individual program pair, not only on average. The PSS counts positions where both sequences match exactly; every such position also constitutes a valid occurrence of the same token in both multisets,

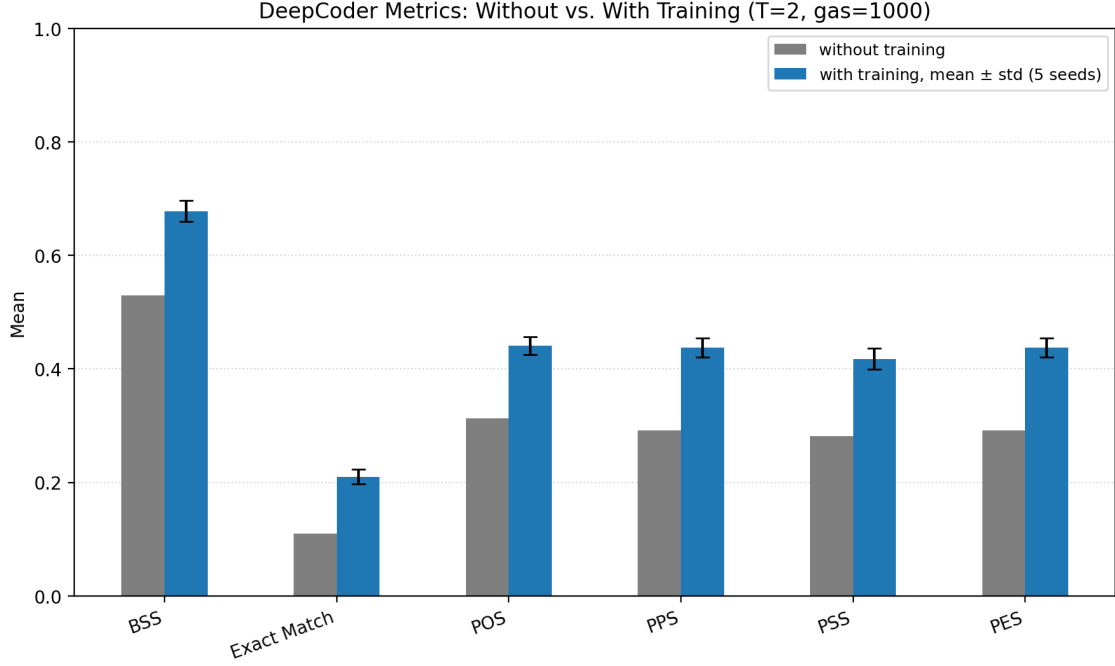


Figure 6.1: DeepCoder: solve rate, exact-match fraction, and the four token-based metrics, without vs. with training ($gas = 1,000$, mean over all 100 tasks).

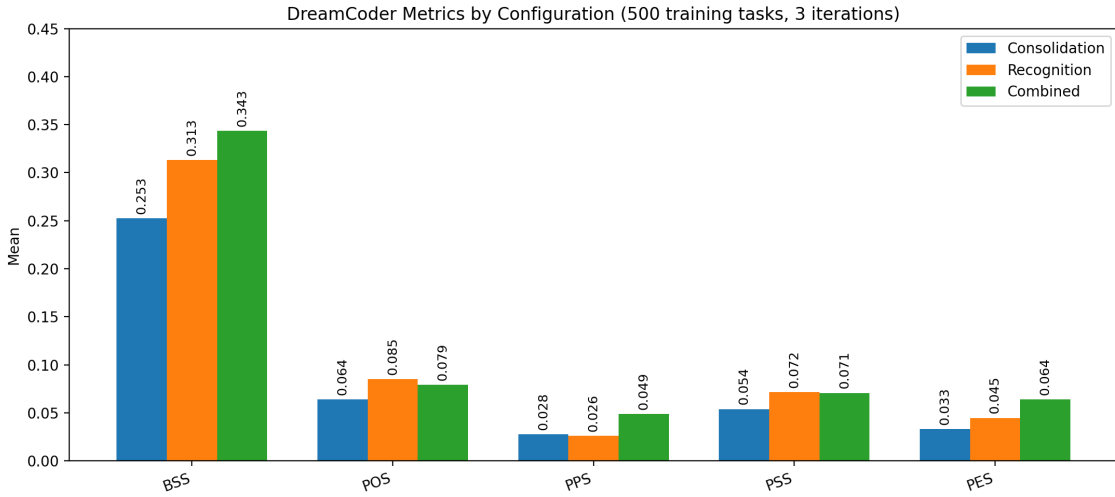


Figure 6.2: DreamCoder: solve rate and the four token-based metrics by configuration (500 training tasks, 3 iterations, mean over all 99 tasks).

so the PPS can never count more matches than the multiset intersection on which the POS is based. The same conclusion holds for the PSS: the longest shared contiguous block uses, of any token involved, at most as many copies as actually occur in both sequences, and is therefore likewise bounded above by the multiset intersection. The

same bound holds for the PES: in every optimal edit sequence between two programs, a set of positions remains unchanged, and its size can be at most as large as the multiset intersection, since every unchanged position is a matching token pair; because the PES is defined only in terms of these unchanged positions minus the insertions or deletions needed to bridge any length difference, the PES too can never exceed the POS. The POS can therefore never lie lower than the PPS, PSS, or PES for any program pair, regardless of how well or poorly a search actually performs.

That the POS leads is therefore not itself an empirical finding about DeepCoder or DreamCoder, since it is guaranteed by the metric definitions; what is empirical is how large the gap to the other three metrics turns out to be in each case. A gap near zero would mean that a search finding the right operations almost always arranges them correctly too; a large gap means the opposite. The previous sections found this gap to differ in size between the two approaches, markedly smaller for DeepCoder than for DreamCoder, and that gap, driven by the training mechanism described earlier rather than by the metrics' guaranteed ranking, is where this pattern's real insight lies.

To rule out that this pattern is a chance finding, three checks test it: whether it is an artifact of the zero convention for unsolved tasks, whether it extends beyond solved tasks to unsolved ones with only partial progress, and whether it holds equally for both approaches rather than being a quirk of just one.

6.3.2 All Tasks versus Solved Tasks

Across all four metrics and both approaches, the mean of each individual metric over solved tasks is consistently higher than its mean over all tasks: roughly one and a half to two times as high for DeepCoder, by a factor of three to four for DreamCoder. This follows directly from the convention that an unsolved task without a generated candidate enters every metric as 0; the mean over all tasks is therefore close to the mean over solved tasks multiplied by the solve rate, and reporting only the all-tasks view would understate how structurally close the search actually gets on the tasks it does solve. Restricting to solved tasks, however, does not change which metric leads: POS remains the highest of the four for every configuration even under this stricter view, so the recurring POS pattern is not an artifact of averaging in the zeros from unsolved tasks. Figure 6.3 isolates the four token-based metrics for DeepCoder over solved tasks only.

These different solve rates also mean different sample sizes behind the solved-only means. DeepCoder solves 53 to 70 of the 100 tasks depending on configuration, DreamCoder only 25 to 34 of the 99. DreamCoder's solved-only means therefore rest on

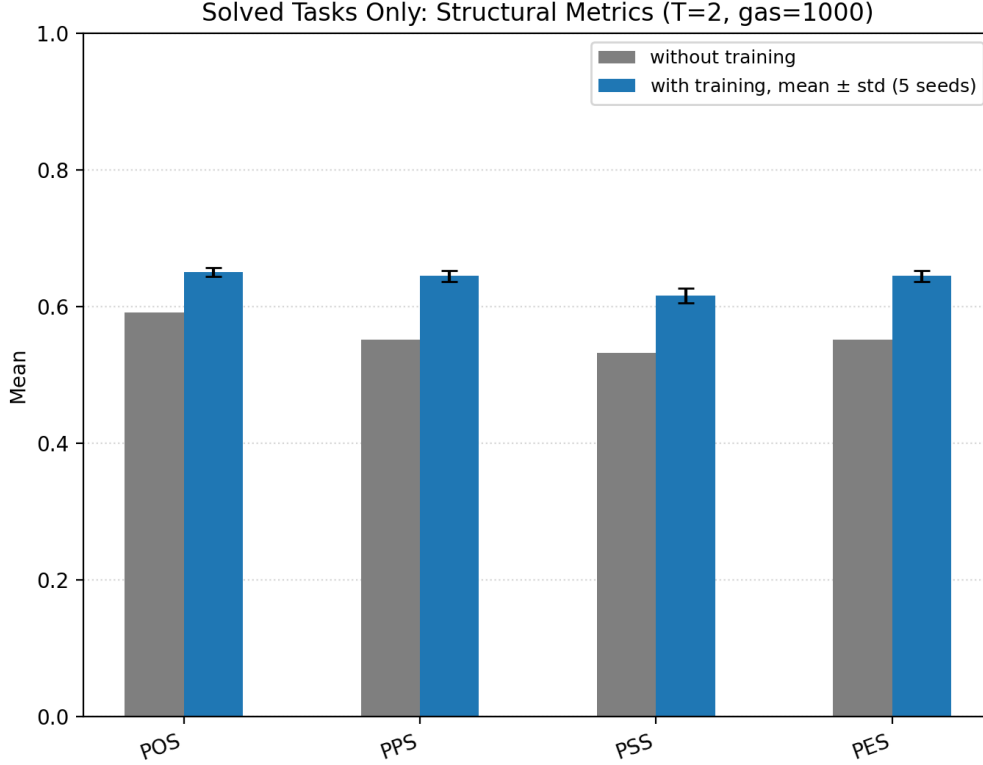


Figure 6.3: DeepCoder: the four token-based metrics without vs. with training, averaged over solved tasks only.

markedly fewer data points than DeepCoder’s, fewest for Consolidation with only 25 solved tasks. Comparisons among the three DreamCoder configurations on the solved side, such as the combined configuration’s lower PSS, therefore rest on a smaller, statistically noisier sample than the corresponding DeepCoder comparisons, one reason to read small differences among the three DreamCoder configurations more cautiously than the considerably larger, more robust gap between the two approaches overall.

The solved side is thus settled for the POS pattern; on the unsolved side, every task so far enters uniformly with the same value 0, regardless of how close a search actually came to a solution. For DeepCoder, this can be made more precise: the best-effort candidates already captured during the search make it possible to assign a value beyond 0 to part of the unsolved tasks as well.

6.3.3 Best-Effort Candidates on Unsolved Tasks

The view restricted to solved tasks is not the whole story on the unsolved side either. For unsolved tasks, DeepCoder’s search additionally captures a *best-effort candidate*: the

partial candidate that satisfied the most of a task’s given examples during the search, even when no candidate solved the task fully. This is the extension to DeepCoder’s search announced as a contribution of this thesis, and it makes it possible to compute the four token-based metrics for a subset of the unsolved tasks instead of uniformly returning 0. Table 6.6 and Figure 6.4 report this per seed, like the rest of this chapter’s with-training results. Because the configuration without training is deterministic, its 47 unsolved tasks and 25 stored best-effort candidates are identical across all five seeds; the solve rate of the configuration with training instead varies per seed, and so does the resulting set of unsolved tasks, which ranges from 30 to 34 tasks, of which 19 to 22 have a stored candidate. Their POS averages 0.265 without training and a mean of 0.302 ± 0.016 with training across the five seeds, clearly above 0 in every seed but also clearly below the corresponding POS of the solved tasks. Unlike the results for solved tasks, the four metrics for these best-effort candidates lie close together in every seed (POS, PPS, PSS, and PES all fall roughly between 0.25 and 0.32), consistent with what a near miss should look like: a handful of scattered token differences rather than the kind of pure reordering that pulls the POS ahead of the other three metrics among solved tasks. On the unsolved side, partial structural progress toward the reference program becomes visible, exactly what a purely binary evaluation cannot show; the remaining tasks in each seed, 22 without training and between 10 and 14 with training, have no usable candidate at all and are excluded from this comparison rather than assigned a misleadingly precise value.

Table 6.6: Best-effort structural metrics on unsolved tasks only, averaged over all unsolved tasks in that seed (tasks without a stored candidate score 0; the count in parentheses is the number of those unsolved tasks with a stored candidate).

Configuration	Unsolved	POS	PPS	PSS	PES
Without training	47 (25)	0.265	0.252	0.252	0.252
With training, mean $\pm$ std	32.2 (20.2)	0.302 ± 0.016	0.280 ± 0.013	0.279 ± 0.013	0.280 ± 0.013

Even among these best-effort candidates, the POS remains on average the highest of the four metrics, just as with solved tasks: the pattern holds for tasks a search does not fully solve as well. Whether it holds equally for DreamCoder, which has no comparable best-effort mechanism, is tested next using the tasks both approaches actually solve.

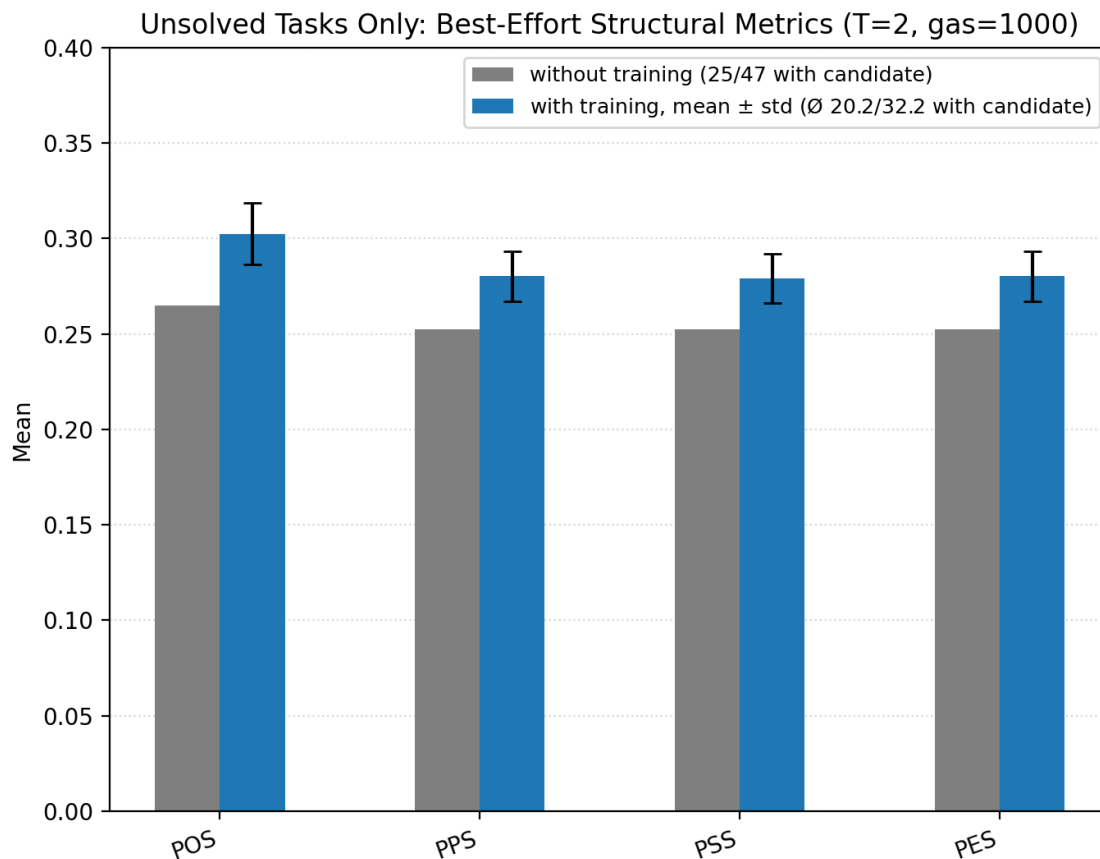


Figure 6.4: DeepCoder: the four token-based metrics for the best-effort candidates of unsolved tasks, without training vs. with training.

6.3.4 DeepCoder versus DreamCoder

Compared directly, DeepCoder scores markedly higher than DreamCoder on all four metrics, in every configuration.

Figure 6.5 shows the same three DreamCoder configurations as Figure 6.2, now averaged over only the solved tasks instead of all 99, and without the BSS itself, which would by definition always equal 1 on this subset.

The recurring POS pattern survives this last check too: it holds for DreamCoder’s three configurations just as it does for DeepCoder’s two, even though the two approaches disagree on almost everything else, absolute values, which configuration performs best, and how strongly the four metrics agree with one another. None of the configurations examined in this chapter breaks the pattern, not even where the other three metrics disagree among themselves about which configuration lies structurally closest to the reference.

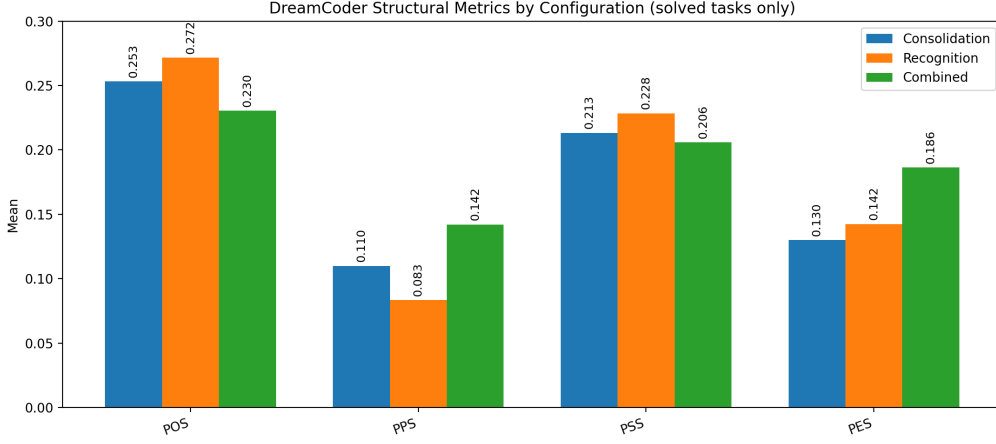


Figure 6.5: DreamCoder: the four token-based metrics by configuration, averaged over solved tasks only.

These aggregate numbers alone, however, say nothing about what a single pair of programs responsible for a particular POS, PPS, PSS, or PES value actually looks like. Three individual cases make this concrete.

6.3.5 Worked Examples

Each of the three cases below establishes its own point. The first shows that even an unsolved task can lie structurally close to the reference, something the BSS alone cannot express. The second shows the reverse case: a solved task whose program lies structurally far from the reference. The third shows why a perfect BSS can still say nothing about actual understanding.

For the DeepCoder task with reference program

LIST|COUNT,<0,0|TAKE,1,0

→ [LIST, COUNT,<0,0, TAKE,1,0],

the first token, LIST, only marks the type of the input parameter, here a single list, not its actual value; it is therefore neither an operation nor the input itself, but a type annotation that comes first in every program of this task family. Only the following two tokens describe the actual computation. The reference program first counts how many elements of the input list are negative, then takes exactly that many elements from the start of the list. Within the search budget, the search found no program that solved all

five given examples. Its best candidate,

$$\text{LIST|FILTER}, <0, 0 | \text{ZIPWITH}, \text{max}, 1, 0$$

$$\rightarrow [\text{LIST}, \text{FILTER}, <0, 0, \text{ZIPWITH}, \text{max}, 1, 0],$$

instead filters out the negative elements and then takes the element-wise maximum of this filtered list and the original list.

Table 6.7: All five given examples of the task from the first example, with the output expected by the reference program and the actual output of the best candidate found.

No.	Input	Expected output (reference)	Found output (candidate)
1	104, 158, -150, -130, -231, -256, 90, -3, -17, 251, -141, 246, 103, -28	104, 158, -150, -130, -231, -256, 90, -3	104, 158, -150, -130, -3, -17, 90, -3
2	-188, 72, -36, -101, 36, -169, 154, 210, 59, -234	-188, 72, -36, -101, 36	-188, 72, -36, -101, 36
3	-86	-86	-86
4	-148, 143, 29, -24, 95, 149, -189, 239, -252, -176, -89, -38, -132, -171, 2, -221, 54, 24	-148, 143, 29, -24, 95, 149, -189, 239, -252, -176	-148, 143, 29, -24, 95, 149, -38, 239, -171, -176
5	29, -46, -153, -105, 232	29, -46, -153	29, -46, -105

Table 6.7 shows both programs on all five given examples: in examples 2 and 3, the candidate happens to match the expected output, in examples 1, 4, and 5 it deviates, for instance in the last example at the final position (-105 instead of -153), which is why the task overall remains unsolved. Built around **FILTER** and **ZIPWITH** instead of **COUNT** and **TAKE**, the candidate agrees with the reference only in its first token, **LIST**. All four metrics agree exactly for this pair, since exactly one of three tokens is present and in the correct position, while the other two are fully replaced:

$$\text{POS} = \text{PPS} = \text{PSS} = \text{PES} = \frac{1}{3}.$$

The task remains unsolved under the BSS, yet a value of roughly a third across all four metrics shows that the candidate is neither arbitrary nor already close to correct: an intermediate result that a pure pass/fail judgment cannot express at all.

The second example shows the reverse case. For the DeepCoder task with reference program

$$\text{LIST|FILTER}, >0, 0 | \text{MAP}, *, -1, 1$$

$$\rightarrow [\text{LIST}, \text{FILTER}, >0, 0, \text{MAP}, * -1, 1],$$

the reference program first filters out all positive elements and then negates the remaining ones. The trained search instead found

$$\text{LIST} | \text{MAP}, * -1, 0 | \text{FILTER}, <0, 1$$

$$\rightarrow [\text{LIST}, \text{MAP}, * -1, 0, \text{FILTER}, <0, 1],$$

which reverses the order: it first negates every element and then keeps only the negative results, the same inversion by which $x > 0$ already becomes $-x < 0$.

Table 6.8: All five given examples of the task from the second example, with the output expected by the reference program and the actual output of the found solution.

No.	Input	Expected output (reference)	Found output (solution)
1	48, 242, -126, -107, -28	-48, -242	-48, -242
2	142, -78, -54, 60	-142, -60	-142, -60
3	19, 161, -74, 63, -114, -5, 40, -36, -243, 113, -52, -163, -8, -161, -241, -149, 141	-19, -161, -63, -40, -113, -141	-19, -161, -63, -40, -113, -141
4	28, -119, 113, 92, 78, 160, 130, -142, 152	-28, -113, -92, -78, -160, -130, -152	-28, -113, -92, -78, -160, -130, -152
5	103, 49, 251, -124, 161, 133, 74, -158, -40, -53, 140, -114, 159, 48, 31, -84, -8	-103, -49, -251, -161, -133, -74, -140, -159, -48, -31	-103, -49, -251, -161, -133, -74, -140, -159, -48, -31

Table 6.8 shows both programs on all five given examples: they produce the same, correct output in every single case, so the task counts as solved. Structurally, however, the two programs share nothing beyond the input token `LIST`, since swapping the order of `FILTER` and `MAP` also shifts their argument indices:

$$\text{POS} = \text{PPS} = \text{PSS} = \text{PES} = \frac{1}{3}.$$

A solved task and a structurally distant program can therefore be the same task, exactly the case the BSS alone cannot distinguish from a solved task whose program stays close to the reference.

The third example is a case where the BSS is right for the wrong reason. For the task with reference program `LIST|LIST|INT|DROP,2,0|ZIPWITH,+,1,3`, whose three leading type markers `LIST`, `LIST`, and `INT` this time mark two lists and an integer as input

parameters, and which drops the first two elements of the first list and adds the result element-wise to the second list, the expected output of all five given examples happens to be the empty list. DreamCoder found the program `(lambda (lambda (lambda (sort empty))))`, which ignores its input entirely and simply returns a constant.

Table 6.9: All five given examples of the task from the third example. Since the found program ignores its input entirely, only the lengths of the two input lists and the integer are given here, not the actual values.

No.	Input (list <i>A</i> , list <i>B</i> , integer)	Expected output	Found output
1	16 elements, 19 elements, 37	empty list	empty list
2	7 elements, 9 elements, 34	empty list	empty list
3	14 elements, 4 elements, 228	empty list	empty list
4	7 elements, 15 elements, 239	empty list	empty list
5	5 elements, 19 elements, 58	empty list	empty list

Table 6.9 shows all five given examples: the inputs differ substantially in length and value, yet the expected output is the empty list in every case, exactly what the found program returns regardless of the actual input. Because the expected outputs of this particular task happen to coincide with what this constant program returns, it satisfies every given example and is recorded as solved with a BSS of 1, even though it does not implement the intended transformation at all. Such programs are flagged separately during evaluation as *trivial solutions* rather than counted among the ordinary solved tasks, precisely because neither the BSS nor the token-based metrics computed on a program that never looks at its input can tell this case apart from a genuine solution on their own. They are rare but not unique: of the 99 test tasks, this affects the same four tasks in each of the three DreamCoder configurations, whose example sets happen to likewise permit a constant output value. More input-output examples per task, or examples deliberately chosen so that a constant output is implausible, would make it harder for this particular program to pass as a solution, but the underlying risk is more general: any evaluation built on a finite set of examples can in principle be satisfied by a program that ignores its input, and a larger example budget only shifts the point at which this becomes unlikely rather than eliminating it.

This third case makes visible that the tools used in this chapter, BSS and token-based metrics alike, run into a fundamental limit as soon as a program satisfies the given examples without implementing the intended rule.

6.4 Limitations

Four limitations qualify how the results above should be read.

Structural closeness is not semantic equivalence. None of the four metrics constitutes a test of semantic equivalence; they measure structural closeness to one specific reference program, not whether two programs compute the same function for all possible inputs. Two structurally very different programs can still be functionally identical, and the metrics have no way to detect this beyond what the BSS already shows on the given test examples, precisely the situation in the second example above, where a solved task scored 0 on both the PPS and the PES. The asymmetry also runs the other way: a single substituted token can already change the computed function on most inputs while barely moving any of the four metrics, so a high value is no more a guarantee of semantic correctness than a low value is a guarantee of semantic difference. The trivial-solution case among the examples above is the most extreme instance of this gap, and at the same time the practical confirmation of the extreme case of underdetermination already named theoretically in Section 3.2: a program that never looks at its input still scored a perfect BSS, because both the BSS and the token-based metrics can only compare behavior on the examples actually given, not on the intended rule behind them. Neither view can overcome this limit on its own, no matter how fine-grained the metric used; it can only be narrowed through additional examples withheld during evaluation, not eliminated by a better metric.

The DreamCoder mapping is incomplete. Comparing DeepCoder and DreamCoder at all requires mapping DreamCoder’s lambda-calculus expressions onto DeepCoder’s DSL vocabulary, and this mapping only covers operation families for which a correspondence was defined; an operation expressed in a way that was not anticipated remains unmapped. Because DreamCoder’s programs are shorter and coarser after this mapping than DeepCoder’s natively tokenized ones, part of the gap reported above between the two approaches reflects the measurement process itself rather than a genuine difference in how close either approach gets to the reference. A more exhaustive mapping, covering additional operation families, could narrow this gap without any change to either approach’s underlying search.

The evaluation covers one domain and one program length. All results in this chapter concern list-processing tasks with reference programs of at most two chained

operations. Longer programs, other domains, or datasets with a different distribution of operations could shift both the solve rates and the relative ordering of the four metrics; the recurring POS pattern in particular has only been tested under this one length restriction. A program-level comparison also inherently depends on how differently two approaches represent programs in the first place, a dependency that a different pair of approaches, with more or less similar representations than DeepCoder and DreamCoder, could weaken or strengthen.

The two approaches are reported with different reproducibility guarantees. DeepCoder’s trained configuration is backed by five independent seeds and reported as a mean with a standard deviation. DreamCoder’s search process also exposes a random-seed parameter, but it only affects a task-batching strategy this thesis does not use, and therefore has no effect on library learning or the recognition model under the configuration used here; every DreamCoder number in this chapter therefore reflects a single run, and its true run-to-run variance remains unknown. The cross-approach comparisons above should be read with this asymmetry in mind: part of an observed gap could, in principle, be explained by variance that DeepCoder’s numbers already show but DreamCoder’s cannot yet show.

What the results of this chapter mean for the research question posed at the outset is drawn together by the next, concluding chapter.

7 Conclusion

This thesis set out from the question of what additional insight into the performance of PBE approaches can be gained through evaluation metrics beyond binary success signals. To answer this question, four token-based metrics were developed, POS, PPS, PSS, and PES, which compare not an approach’s output but the program it generates itself to a reference program, and applied to two structurally fundamentally different approaches: DeepCoder, which searches over a fixed DSL, and DreamCoder, which extends its own library during the search. Because a direct token comparison between DeepCoder’s DSL programs and DreamCoder’s lambda-calculus expressions is not possible without further work, this first required mapping both program representations onto a shared operation vocabulary.

The four metrics each isolate a different kind of structural deviation. The POS checks whether the right operations are present at all, regardless of their position. The PPS checks whether they sit exactly in the right place. The PSS checks whether they at least survive as a contiguous block. The PES finally condenses all three aspects into a single number of required edits. Read together with the BSS, the four metrics allow an interpretation that neither view could provide on its own.

Across five configurations examined, DeepCoder without and with a trained attribute predictor over five seeds (solve rate from 53.0% to $67.8\% \pm 1.8\%$) as well as DreamCoder’s three learning mechanisms Consolidation, Recognition, and their combination (solve rate between 25.3% and 34.3%), a consistent pattern emerged: the POS was, without exception, higher than the other three metrics. Both approaches, in other words, find the right building blocks of a solution more often than they arrange those building blocks correctly. This pattern proved robust across three independent checks: it held under the stricter view of solved tasks only, under DeepCoder’s best-effort candidates on tasks no search fully solved, and for DreamCoder just as for DeepCoder, even though the two approaches differed in practically every other respect.

That a program solves the given examples rarely means it matches the reference: only between 11.0% and 21.0% of all DeepCoder tasks were matched exactly, token for

token, even though considerably more count as solved. Three worked individual cases made visible what this means in practice. A best-effort candidate that matched only one of three reference tokens still received a value of roughly a third across all four metrics, a gradation a pure pass/fail judgment cannot express. A solved DreamCoder task, conversely, reached a PPS and PES of 0 each, because its solution produced the same output by a structurally completely different route. A third example finally showed the limit of both views at once: a program that ignored its input entirely happened to satisfy the given examples and thereby received a perfect BSS, even though it did not implement the intended transformation.

This makes it possible to answer the research question posed at the outset. A BSS conceals how close a wrong solution actually comes to the reference, and whether a solved task was satisfied by a structurally close or a structurally distant solution. Program-level metrics close this gap by showing what proportion of a solution is already correct and in what respect it deviates from the reference, in the presence of the right operations, in their position, or in their contiguity, and whether the same kind of deviation recurs across configurations and even across structurally different approaches. They are most informative precisely where the BSS distinguishes least: on unsolved tasks with partial progress, and on solved tasks whose solution deviates structurally from the reference.

These findings hold within the limitations named in the previous chapter: they rest on a single domain and a length restriction to two operations, on an incomplete mapping between DreamCoder’s and DeepCoder’s program representations, and on structural rather than semantic closeness, which cannot guarantee the functional equality of two programs.

7.1 Future Work

These limitations also point to natural directions for future work. Most immediately, one could test whether the recurring POS pattern persists beyond the list-processing domain and the two-operation length restriction examined here, for instance in the text domain or with longer reference programs, where more room exists for reordering and partial correctness.

A more exhaustive mapping between DreamCoder’s lambda-calculus expressions and DeepCoder’s DSL vocabulary, covering additional operation families, could also attribute a larger share of the observed gap between the two approaches to actual search success rather than to the measurement process itself. Likewise, the best-effort mechanism

introduced for DeepCoder in this thesis could be transferred to DreamCoder, to make the insights gained on the unsolved side available for both approaches rather than only one.

Finally, structural closeness remains no substitute for a proof of semantic equivalence. An extension that combines the token-based metrics developed in this thesis with a check of functional equality on additional inputs withheld during evaluation could further close the demonstrated gap between structural and semantic correctness. This would allow the movement beyond the BSS begun in this thesis to be carried forward consistently.

References

- [1] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. Deepcoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1611.01989>.
- [2] José Cambronero, Sumit Gulwani, et al. Flashfill++: Scaling programming by example by cutting to the chase. *Proc. ACM Program. Lang.*, 7(POPL):33, 2023.
- [3] Mark Chen, Jerry Tworek, et al. Evaluating large language models trained on code, 2021.
- [4] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3):171–176, 1964.
- [5] Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. Robustfill: Neural program learning under noisy i/o. In *Proceedings of the 34th International Conference on Machine Learning*, 2017. URL <https://arxiv.org/abs/1703.07469>.
- [6] Kevin Ellis, Maxwell Nye, et al. Write, execute, assess: Program synthesis with a repl. In *33rd Conference on Neural Information Processing Systems (NeurIPS 2019)*, 2019.
- [7] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. Dream-coder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning, 2020. URL <https://arxiv.org/abs/2006.08381>.
- [8] Kevin Ellis et al. ec (official dreamcoder implementation). GitHub repository. URL <https://github.com/ellisk42/ec>.

-
- [9] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '11)*, 2011.
 - [10] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 2017.
 - [11] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. In *1st Mathematical Reasoning in General Artificial Intelligence Workshop, ICLR 2021*, 2021. URL <https://github.com/hendrycks/apps>.
 - [12] David Kamm. deepcoder (unofficial deepcoder reference implementation). GitHub repository. URL <https://github.com/dkamm/deepcoder>.
 - [13] Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938. doi: 10.2307/2332226.
 - [14] Tessa A. Lau, Steven A. Wolfman, Pedro Domingos, and Daniel S. Weld. Programming by demonstration using version space algebra. *Machine Learning*, 53:111–156, 2003.
 - [15] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.
 - [16] Wen-Ding Li and Kevin Ellis. Is programming by example solved by llms? In *38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024.
 - [17] Chin-Yew Lin and Franz Josef Och. Automatic evaluation of machine translation quality using longest common subsequence and skip-bigram statistics. In *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics*, 2004. doi: 10.3115/1218955.1219032.
 - [18] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002. doi: 10.3115/1073083.1073135.

-
- [19] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, 2015. URL <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/oopsla15-pbe.pdf>.
 - [20] Oona Rainio, Jarmo Teuho, and Riku Klén. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14:6086, 2024. doi: 10.1038/s41598-024-56706-x.
 - [21] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. Codebleu: a method for automatic evaluation of code synthesis, 2020. URL <https://arxiv.org/abs/2009.10297>.
 - [22] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of nlp models with checklist. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. URL <https://aclanthology.org/2020.acl-main.442/>.
 - [23] Tobias Sesterhenn, Ian Berlot-Attwell, Janis Zenkner, and Christian Bartelt. A compute-matched re-evaluation of trove on math. In *The Second AI for MATH Workshop at ICML 2025*, 2025.
 - [24] Kensen Shi, Joey Hong, Yinlin Deng, Pengcheng Yin, Manzil Zaheer, and Charles Sutton. Exedec: Execution decomposition for compositional generalization in neural program synthesis. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2307.13883>.
 - [25] Yewei Song, Cedric Lothritz, Daniel Tang, Tegawendé F. Bissyandé, and Jacques Klein. Revisiting code similarity evaluation with abstract syntax tree edit distance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2024. URL <https://aclanthology.org/2024.acl-short.3/>.
 - [26] Bo Wang, Teodora Baluta, Aashish Kolluri, and Prateek Saxena. Synguar: Guaranteeing generalization in programming by example. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021. doi: 10.1145/3468264.3468621.

-
- [27] Anjiang Wei, Tarun Suresh, Jiannan Cao, Naveen Kannan, Yuheng Wu, Kai Yan, Thiago S. F. X. Teixeira, Ke Wang, and Alex Aiken. Codearc: Benchmarking reasoning capabilities of llm agents for inductive program synthesis, 2025. URL <https://arxiv.org/abs/2503.23145>.
 - [28] Steve Leonel Yomi Mbiakop. deepcoder (fork with the extensions and evaluation pipeline developed for this thesis). GitHub repository, 2026. URL <https://github.com/YOMILEONEL/deepcoder>.
 - [29] Steve Leonel Yomi Mbiakop. ec (fork with the extensions and evaluation pipeline developed for this thesis). GitHub repository, 2026. URL <https://github.com/YOMILEONEL/ec>.
 - [30] Janis Zenkner, Tobias Sesterhenn, and Christian Bartelt. Shedding light on task decomposition in program synthesis: The driving force of the synthesizer model. In *Published as a workshop paper at ICLR 2025*, 2025. URL <https://arxiv.org/abs/2503.08738>.
 - [31] Jiasheng Zheng, Boxi Cao, Zhengzhao Ma, Ruotong Pan, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. Beyond correctness: Benchmarking multi-dimensional code generation for large language models, 2024. URL <https://arxiv.org/abs/2407.11470>.